arXiv:2601.21363v3 [cs.RO] 21 Feb 2026

TOWARDS BRIDGING THE GAP BETWEEN LARGE-SCALE PRETRAINING AND EFFICIENT FINETUNING FOR HUMANOID CONTROL

Weidong Huang[1], **Zhehan Li**[1,2], **Hangxin Liu**[1], **Biao Hou**[2], **Yao Su**[1], **Jingwen Zhang**[1*]
[1]State Key Laboratory of General Artificial Intelligence, BIGAI
[2]School of Artificial Intelligence, Xidian University

ABSTRACT

Reinforcement Learning (RL) is widely used for humanoid control, with on-policy methods such as Proximal Policy Optimization (PPO) enabling robust training via large-scale parallel simulation and, in some cases, zero-shot deployment to real robots. However, the low sample efficiency of on-policy algorithms limits safe adaptation to new environments. Although off-policy RL and model-based RL have shown improved sample efficiency, the gap between large-scale pretraining and efficient finetuning on humanoids still exists. In this paper, we find that off-policy Soft Actor-Critic (SAC), with large-batch update and a high Update-To-Data (UTD) ratio, reliably supports large-scale pretraining of humanoid locomotion policies, achieving zero-shot deployment on real robots. For adaptation, we demonstrate that these SAC-pretrained policies can be finetuned in new environments and out-of-distribution tasks using model-based methods. Data collection in the new environment executes a deterministic policy while stochastic exploration is instead confined to a physics-informed world model. This separation mitigates the risks of random exploration during adaptation while preserving exploratory coverage for improvement. Overall, the approach couples the wall-clock efficiency of large-scale simulation during pretraining with the sample efficiency of model-based learning during fine-tuning. **Code and videos: `https://lift-humanoid.github.io`**

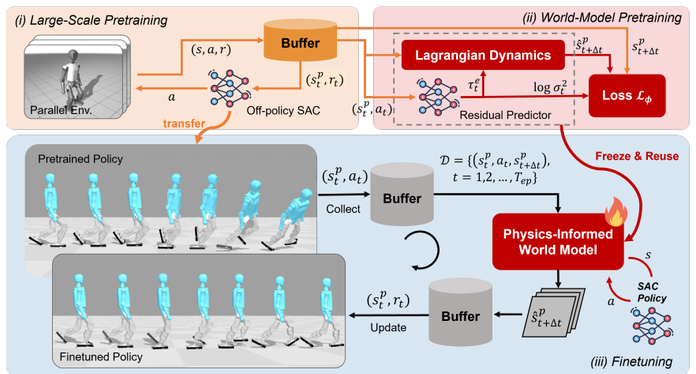


Figure 1: **L**arge-scale pretra**I**ning and efficient **F**ine**T**uning (LIFT) Framework. In stage (i), we implement SAC in JAX to support large-batch update and high UTD, achieving fast, robust convergence in massively parallel simulation and zero-shot deployment to a real humanoid in outdoor experiments. In stage (ii), we pretrain a physics-informed world model on the SAC data, combining Lagrangian dynamics with a residual predictor to capture contact forces and other unmodeled effects. In stage (iii), we finetune both the policy and the world model to new environments while executing only deterministic actions in the environment. Stochastic exploration is confined to rollouts within the world model. This framework enhances both the safety and efficiency of finetuning.

*Corresponding Author.

arXiv:2601.21363v3 [cs.RO] 21 Feb 2026

迈向弥合人形控制中大规模预训练与高效微调之间的鸿沟

黄卫东[1], **李哲瀚**[1,2], **刘航新**[1], **侯彪**[2], **苏 Yao**[1], **张静文**[1*]通用人工智能国家重点实验室，BIGAI[2]西安电子科技大学人工智能学院

摘要

强化学习（RL）广泛应用于人形控制，其中同策略方法如近端策略优化（PPO）通过大规模并行仿真实现了鲁棒训练，并在某些情况下支持零样本部署到真实机器人。然而，同策略算法的低样本效率限制了其在新环境中的安全适应。尽管离线强化学习和基于模型的强化学习已展现出更高的样本效率，但在人形机器人上，大规模预训练与高效微调之间仍存在差距。本文发现，采用大批量更新和高更新 - 数据比（UTD）的异策略软演员 - 评论家（SAC）算法，能够可靠地支持人形运动策略的大规模预训练，并实现在真实机器人上的零样本部署。针对适应性问题，我们证明这些经 SAC 预训练的策略可利用基于模型的方法在新环境和分布外任务中进行微调。在新环境中的数据收集执行确定性策略，而随机探索则被限制在物理信息世界模型内。这种分离方式在保留改进所需探索覆盖范围的同时，降低了适应过程中随机探索的风险。总体而言，该方法将预训练阶段大规模仿真的实际时间效率与微调阶段基于模型学习的样本效率相结合。**代码和视频：`https://lift-humanoid.github.io`**

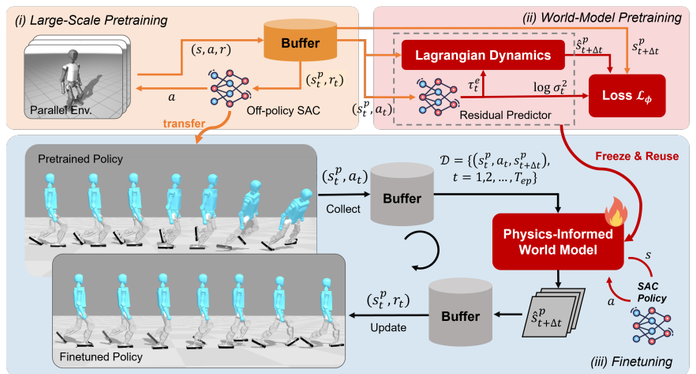


图 1：**L**arge-scale pretra**I**ning and efficient**F**ine**T**uning (LIFT) 框架。在阶段 (i) 中，我们在 JAX 中实现 SAC 以支持大批量更新和高 UTD，从而在大规模并行模拟中实现快速、鲁棒的收敛，并在户外实验中零样本部署到真实人形机器人上。在阶段 (ii) 中，我们在 SAC 数据上预训练一个物理信息世界模型，结合拉格朗日动力学与残差预测器，以捕捉接触力及其他未建模效应。在阶段 (iii) 中，我们将策略和世界模型微调至新环境，同时在环境中仅执行确定性动作。随机探索仅限于世界模型内的 rollout。该框架提升了微调的安全性和效率。

*通讯作者。

1 INTRODUCTION

Learning generalist physical agents is a long-standing goal in AI, and humanoid robots offer broad task coverage among robotic embodiments. PPO (Schulman et al., 2017) becomes a mainstream baseline for humanoid robot control due to its robustness and fast wall-clock convergence in massively parallel GPU simulation (Schwarke et al., 2025). However, when only limited real-world or cross-task data can be collected, on-policy methods are disadvantaged because they discard off-policy experience (Haarnoja et al., 2018b; Lillicrap et al., 2015; Fujimoto et al., 2018). Moreover, even when a PPO policy achieves zero-shot deployment in new environments, performance metrics can degenerate (Gu et al., 2024) (e.g., poorer velocity tracking or reduced execution precision). These limitations highlight the need for frameworks that can reuse prior experience and adapt policies efficiently in new environments.

Off-policy algorithms offer an appealing backbone for this paradigm because they can leverage replayed experience for sample-efficient finetuning (Lillicrap et al., 2015). However, these algorithms tend to overfit to replayed data, which can bias exploration during fine-tuning, especially when large-scale parallel simulation with them has received limited attention. During fine-tuning, directly adapting a policy to a new environment often leads to unsafe actions because of stochastic exploration. This risk is especially high for humanoid robots: small support polygons, particularly during single-support phases, make the system highly sensitive to perturbations.

Constraining exploration within a learned world model provides a safer and more sample-efficient alternative (Levy et al., 2024; Hafner et al., 2023), though synthetic rollouts may introduce model bias that destabilizes finetuning (Ha & Schmidhuber, 2018; Janner et al., 2019). A physics-informed world model (Levy et al., 2024) that incorporates structural priors from physcis improves the fidelity of synthetic rollouts, reduces model bias, and enables more reliable finetuning even with limited data. However, training such model-based systems from scratch (Levy et al., 2024) remains extremely slow in wall-clock time and prone to local minima. Meanwhile, we find that off-policy SAC (Haarnoja et al., 2018b) integrates more naturally with model-based algorithms (Levy et al., 2024) than PPO, achieving robust and sample-efficient improvements. These findings further motivate our pipeline. We introduce **L**arge-scale pretra**I**ning and efficient **F**ine**T**uning (**LIFT**), a three-stage framework as shown in the Figure 1: (i) large-scale policy pretraining, (ii) physics-informed world model pretraining, and (iii) efficient finetuning of the policy and world model. We make the following contributions:

1. We provide a scalable JAX implementation of SAC that supports robust convergence in massively parallel simulation and zero-shot deployment to a physical humanoid robot within one hour of wall-clock training on a single NVIDIA RTX 4090 GPU. The resulting SAC policy also serves as the policy module within our model-based finetuning stage.
2. We develop a fine-tuning strategy that executes deterministic actions in new environments while limiting stochastic exploration in a physics-informed world model, which improves sample efficiency and stabilizes convergence. This physics-informed design is pivotal for finetuning, enabling data-efficient in-distribution adaptation and stronger out-of-distribution generalization.
3. We release an open-source pipeline for humanoid control spanning pretraining, zero-shot deployment, and finetuning, providing a practical baseline for the robotics community.

2 RELATED WORKS

Large-Scale RL Pretraining in Simulation. With the rapid progress of GPU technology, several GPU-based physics simulators have emerged, such as IsaacGym (Makoviychuk et al., 2021), Mu-joco Playground (Zakka et al., 2025), and Brax (Freeman et al., 2021). These frameworks enable fast parallel simulation of thousands of robot environments directly on GPUs. PPO (Schulman et al., 2017) has become the dominant baseline in this setting due to the ease of implementation and robust convergence. By leveraging domain randomization (Tobin et al., 2017), PPO policies can achieve fast wall-clock convergence and even zero-shot transfer to physical humanoid robots (Rudin et al., 2022; Gu et al., 2024). However, the low sample efficiency of on-policy methods limits their ability to adapt or continue training in new environments.

1 引言

学习通用物理智能体是人工智能领域长期以来的目标，而人形机器人在各类机器人实体中提供了最广泛的任务覆盖范围。PPO（Schulman 等人，2017）因其在大规模并行 GPU 仿真（Schwarke 等人，2025）中表现出的鲁棒性和快速的挂钟时间收敛性，已成为人形机器人控制的主流基线方法。然而，当仅能收集有限的真实世界或跨任务数据时，同策略方法处于劣势，因为它们会丢弃离策略经验（Haarnoja 等人，2018b；Lillicrap 等人，2015；Fujimoto 等人，2018）。此外，即使 PPO 策略在新环境中实现了零样本部署，其性能指标也可能退化（Gu 等人，2024）（例如，速度跟踪能力变差或执行精度降低）。这些局限性凸显了亟需开发能够复用先前经验并在新环境中高效适应策略的框架。

离线策略算法为这一范式提供了极具吸引力的基础，因为它们能够利用重放的经验进行样本高效微调（Lillicrap 等人，2015）。然而，这些算法往往会对重放数据过拟合，从而在微调过程中使探索产生偏差，尤其是当与之相关的大规模并行仿真尚未得到充分关注时。在微调期间，由于随机探索，直接将策略适应新环境往往会导致不安全的动作。对于人形机器人而言，这种风险尤其高：较小的支撑多边形，特别是在单支撑阶段，使得系统对扰动高度敏感。

在学习得到的世界模型内约束探索提供了一种更安全且样本效率更高的替代方案（Levy 等人，2024；Hafner 等人，2023），尽管合成 rollout 可能会引入模型偏差，从而导致微调不稳定（Ha 与 Schmidhuber，2018；Janner 等人，2019）。一种融合了物理结构先验的物理信息世界模型（Levy 等人，2024）提高了合成 rollout 的保真度，减少了模型偏差，并使得即使在数据有限的情况下也能实现更可靠的微调。然而，从头训练此类基于模型的系统（Levy 等人，2024）在挂钟时间上仍然极其缓慢，且容易陷入局部极小值。与此同时，我们发现异策略 SAC（Haarnoja 等人，2018b）比 PPO 能更自然地与基于模型的算法（Levy 等人，2024）集成，从而实现稳健且样本高效的改进。这些发现进一步激发了我们构建该流程的动力。我们提出了**L**arge-scale pretra**I**ning a nd efficient **F**ine**T**uning（**LIFT**），这是一个如图 1 所示的三阶段框架：(i) 大规模策略预训练，(ii) 物理信息世界模型预训练，以及 (iii) 策略和世界模型的高效微调。我们的贡献如下：

1. 我们提供了一个可扩展的 JAX SAC 实现，支持在大规模并行模拟中实现鲁棒收敛，并能零样本部署到物理人形机器人上
   仅需在单块 NVIDIA RTX 4090 GPU 上进行一小时的挂钟时间训练。所得的 SAC 策略也可作为我们基于模型的微调阶段中的策略模块。
2. 我们开发了一种微调策略，该策略在新环境中执行确定性动作，同时在物理信息世界模型中限制随机探索，从而提高了样本效率并稳定了收敛过程。这种物理信息设计对于微调至关重要，能够实现数据高效的分布内适应和更强的分布外泛化能力。
3. 我们发布了一个用于人形控制的开源流水线，涵盖预训练、零样本部署和微调，为机器人学社区提供了一个实用的基线。

2 相关工作

大规模强化学习预训练在仿真中。随着 GPU 技术的快速发展，出现了多种基于 GPU 的物理模拟器，例如 IsaacGym（Makoviychuk 等，2021）、Mu-joco Playground（Zakka 等，2025）和 Brax（Freeman 等，2021）。这些框架能够在 GPU 上直接实现数千个机器人环境的快速并行模拟。由于易于实现且具有鲁棒收敛性，PPO（Schulman 等人，2017）已成为该场景下的主流基线方法。通过利用域随机化（Tobin 等，2017），PPO 策略可以实现快速的挂钟时间收敛，甚至能够零样本迁移到实体人形机器人上（Rudin 等，2022；Gu 等人，2024）。然而，同策略方法的低样本效率限制了其在新环境中适应或继续训练的能力。

Off-Policy RL. When data collection is expensive or risky, improving sample efficiency becomes critical. Off-policy algorithms such as TD3 (Fujimoto et al., 2018) and SAC (Haarnoja et al., 2018b) achieve higher sample efficiency by reusing past experiences and leveraging critic gradients to update the policy. SAC has been used to train quadruped robots from scratch with limited real-world data (Haarnoja et al., 2018c; Ha et al., 2020; Haarnoja et al., 2018a; Smith et al., 2022b), and pretrained SAC policies can be finetuned on hardware with the same algorithm (Smith et al., 2022a; Ball et al., 2023). However, these methods typically rely on stochastic exploration in the environment, where injected action noise may damage actuators or induce unsafe states. This risk is amplified for humanoids, whose smaller support polygons (especially in single-support phases) make them sensitive to perturbations compared to quadrupeds. These methods also underutilize large-scale parallel simulation, resulting in high wall-clock training time. Recent large-scale off-policy efforts include Parallel Q-Learning (Li et al., 2023), which scales DDPG (Lillicrap et al., 2015) across massive simulations yet without sim-to-real validation, and FastTD3 (Seo et al., 2025), which achieves humanoid sim-to-real but its finetuning ability in new environments remains unclear. Raffin (2025) show that SAC can be made stable in massively parallel simulators through extensive parameter tuning, but their work also lacks sim-to-real evaluation and does not examine fine-tuning performance.

Model-Based Techniques Model-based methods can further improve sample efficiency, such as MBPO (Janner et al., 2019) using synthetic rollouts and Dreamer (Hafner et al., 2023) showing strong data efficiency in real-world tasks (Wu et al., 2023). However, they often still require stochastic exploration in the environment. Sun et al. (2025) train a world model that explicitly reconstructs the environment state and use it as policy input to enhance locomotion robustness, but the model is not used to generate synthetic data for policy fine-tuning aimed at improving adaptability. ASAP (He et al., 2025) collects real-world data using a pretrained policy and learns a delta-action model to correct simulator actions before fine-tuning. However, the delta network can output unbounded action corrections while finetuning, which may lead to undesired behaviors, and the approach requires substantial real-world data to sufficiently cover the gaps between the simulator and the physical robot, especially across diverse motions. These limitations highlight an open question: how can physics priors be incorporated into world models to improve predictive accuracy while reducing the amount of data required for policy fine-tuning? Physics-Informed Neural Networks (PINNs) (Raissi et al., 2019) represent one possible approach by embedding physical constraints directly into the learning architecture. For example, Greydanus et al. (2019) proposed Hamiltonian neural networks that learn energy-conserving dynamics in an unsupervised manner, while Cranmer et al. (2020) introduced Lagrangian neural networks that model dynamics without requiring canonical coordinates. Deep Lagrangian Networks (Lutter et al., 2019) further demonstrated strong performance in robot tracking and extrapolation to novel trajectories. Yet these approaches remain limited to energy-conserving systems and struggle with contact-rich dynamics. To address this, SSRL (Levy et al., 2024) proposed combining rigid-body dynamics with learned contact-force models, enabling a quadruped robot to learn to walk with just three minutes of real-world data. Nevertheless, training from scratch remains unsafe for humanoids due to their fragility and instability. Motivated by this, our work adopts a pretrain–finetune pipeline that extends physics-informed modeling to large-scale humanoid pretraining and safe finetuning.

3 PRELIMINARIES

Problem Formulation. RL is typically modeled as a Markov Decision Process (MDP) (Altman, 1999): $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, R, \mu, \gamma)$, where $\mathcal{S}$ and $\mathcal{A}$ denote the state and action spaces, respectively. $\mathbb{P}(s'|s,a)$ and $R(r|s,a)$ represent the transition probability and reward function. $\mu(\cdot)$ denotes the initial state distribution, and γ is the discount factor. A stochastic policy π_θ parameterized by θ specifies the action probability $\pi_\theta(a|s)$ for a given state s. In continuous control, the stochastic policy is typically Gaussian, $\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s), \Sigma_\theta(s)\big)$, where the covariance can be either state-independent or state-dependent. During pretraining, domain randomization is employed to enhance robustness by perturbing the environment transition function $\mathbb{P}$ across episodes. At the same time, the stochastic policy is used to promote diverse exploration. For the deployment and finetuning stage, however, we adopt the deterministic policy induced by the mean action $\mu_\theta(s)$ to collect data and evaluate performance in environments.

离线强化学习。当数据收集成本高昂或存在风险时，提高样本效率变得至关重要。离线策略算法（如 TD3 （ Fujimoto 等人，2018 ）和 SAC （ Haarnoja 等人，2018b ））通过复用过往经验并利用评论家梯度来更新策略，从而实现了更高的样本效率。SAC 已被用于在有限的真实世界数据下从头训练四足机器人（Haarnoja 等，2018c；Ha 等，2020；Haarnoja 等，2018a；Smith 等，2022b），并且预训练的 SAC 策略可以使用相同算法在硬件上进行微调（Smith 等，2022a；Ball 等，2023）。然而，这些方法通常依赖于环境中的随机探索，其中注入的动作噪声可能会损坏执行器或引发不安全状态。对于人形机器人而言，这种风险被进一步放大，因为其支撑多边形较小（尤其是在单支撑阶段），使其相比四足机器人对扰动更为敏感。此外，这些方法未能充分利用大规模并行仿真，导致挂钟时间训练耗时较长。近期的大规模离线策略努力包括并行 Q 学习（Li 等，2023），该方法将 DDPG（Lillicrap 等人，2015）扩展至海量仿真中，但缺乏仿真到现实的验证；以及 FastTD3（Seo 等，2025），该方法实现了人形机器人的仿真到现实迁移，但其在新环境中的微调能力尚不明确。Raffin（2025）表明，通过广泛的参数调优，SAC 可以在大规模并行模拟器中保持稳定，但其工作同样缺乏仿真到现实的评估，也未考察微调性能。

基于模型的方法基于模型的方法可以进一步提高样本效率，例如 MBPO（Janner 等人，2019）利用合成 rollout，以及 Dreamer（Hafner 等人，2023）在现实世界任务中展现出强大的数据效率（Wu 等，2023）。然而，它们通常仍需要在环境中进行随机探索。Sun 等（2025）训练了一个显式重构环境状态的世界模型，并将其用作策略输入以增强运动鲁棒性，但该模型并未用于生成合成数据以进行旨在提升适应性的策略微调。ASAP（He 等，2025）利用预训练策略收集现实世界数据，并学习一个增量动作模型，以便在微调前校正模拟器动作。然而，该增量网络在微调过程中可能输出无界的动作校正，从而导致不良行为；此外，该方法需要大量现实世界数据才能充分覆盖模拟器与物理机器人之间的差距，尤其是在多种运动模式下。这些局限性凸显了一个开放性问题：如何将物理先验融入世界模型，以提高预测精度并减少策略微调所需的数据量？物理信息神经网络（PINNs）（Raissi 等，2019）代表了一种可能的途径，即通过将物理约束直接嵌入学习架构来实现。例如，Greydanus 等（2019）提出了哈密顿神经网络，能够以无监督方式学习能量守恒动力学；而 Cranmer 等（2020）引入了拉格朗日神经网络，可在无需规范坐标的情况下对动力学进行建模。深度拉格朗日网络（Lutter 等，2019）进一步在机器人跟踪及对新轨迹的外推方面展现了优异性能。然而，这些方法仍局限于能量守恒系统，难以处理富含接触的动力学场景。为解决这一问题，SSRL（Levy 等人，2024）提出将刚体动力学与学习到的接触力模型相结合，使四足机器人仅凭三分钟的现实世界数据即可学会行走。尽管如此，由于人形机器人的脆弱性和不稳定性，从头开始训练仍然存在安全风险。受此启发，我们的工作采用了一种预训练 - 微调流程，将物理信息建模扩展至大规模人形机器人预训练与安全微调。

3 预备知识

问题表述。强化学习（RL）通常被建模为马尔可夫决策过程（MDP）（Altman, 1999）：$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathbb{P}, R, \mu, \gamma)$，其中 $\mathcal{S}$ 和 $\mathcal{A}$ 分别表示状态空间和动作空间。$\mathbb{P}(s'|s,a)$ 和 $R(r|s,a)$ 代表转移概率和奖励函数。$\mu(\cdot)$ 表示初始状态分布，γ 是折扣因子。一个由 θ 参数化的随机策略 π_θ 规定了给定状态 s 下的动作概率 $\pi_\theta(a|s)$。在连续控制中，随机策略通常采用高斯分布，$\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s)\ \Sigma_\theta(s)\big),$，其协方差可以是状态无关的或状态相关的。在预训练期间，采用域随机化技术，通过在多个回合中扰动环境转移函数 $\mathbb{P}$ 来增强鲁棒性。同时，使用随机策略以促进多样化的探索。然而，在部署和微调阶段，我们采用由平均动作 $\mu_\theta(s)$ 诱导的确定性策略，以收集数据并在环境中评估性能。

Model-based RL Problem. For the finetuning stage, we seek a policy $\pi_\theta \in \Pi_\theta$ that maximizes the model-predicted return $J^R_\phi(\pi_\theta)$, defined as follows:

$$J^R_\phi(\pi_\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \,\middle|\, s_0 \sim \mu,\ a_t \sim \pi_\theta(\cdot|s_t),\ (s_{t+1}, r_{t+1}) \sim \mathbb{P}_\phi(\cdot \mid s_t, a_t)\right]. \tag{1}$$

Here $\mathbb{P}_\phi(\cdot|s_t, a_t)$ denotes a world model parameterized by ϕ, which approximates the true environment dynamics. The initial state s_0 is sampled from the true initial distribution μ, while subsequent states are generated by the learned model. In practice, a stochastic policy π_θ is used to interact with $\mathbb{P}_\phi$ so as to generate diverse imaginary trajectories, thereby improving exploration coverage for policy optimization. When the predictions of the world model are sufficiently accurate, maximizing the model-predicted return corresponds to maximizing the return in the real environment (Janner et al., 2019; Hafner et al., 2023).

Physics-Informed World Model for Robots Incorporating physics priors into the world model improves the accuracy of its predictions (Raissi et al., 2019). We adopt a physics-informed formulation of robot dynamics based on the Lagrangian equations of motion (Featherstone, 2008):

$$M(q_t)\ddot{q}_t + C(q_t, \dot{q}_t) + G(q_t) = B\tau_t + J^\top F^e_t + \tau^d_t, \tag{2}$$

Here, q_t and $\dot{q}_t$ are the generalized coordinates and velocities at time t. The motor torque τ_t is applied to the joints via the matrix B. F^e_t is the external contact force with Jacobian J, and τ^d_t denotes dissipative (parasitic) torques. The matrices M, C, and G are the mass, Coriolis/centrifugal, and gravity terms. Because B, M, C, and G depend only on the robot's shape and inertial parameters, they are known beforehand. Thus, the main model uncertainties are the contact term $J^\top F^e_t$ and the dissipative term τ^d_t. To model these unknown contributions, Levy et al. (2024) introduce a residual network that approximates them. The network predicts the sum of the contact term and the dissipative term, as well as the uncertainty of the next-state prediction:

$$\tau_\phi(s_t, a_t) \approx \tau^e_t \equiv J^\top F^e_t + \tau^d_t \tag{3}$$

$$\sigma_\phi(s_t, a_t) = \log \sigma^2_t \tag{4}$$

Here, τ_ϕ and σ_ϕ are outputs from different heads of the neural network, with σ^2_t representing the predicted variance of the next state. This hybrid physics-informed world model $\mathbb{P}_\phi$ combines the known rigid-body dynamics with learned residuals and can be rolled out to generate more precise trajectories for policy optimization.

4 Large-scale Pretraining and Efficient Fine-tuning

To enable a humanoid robot to learn quickly and with high sample efficiency in new environments, we first select the algorithm best suited for fine-tuning and then design the framework around it. Using a single algorithm across both stages aligns training objectives, enables continuous policy updates that mitigate forgetting, and provides a unified, easy-to-implement framework. While SSRL (Levy et al., 2024) successfully trained a quadruped robot from scratch using SAC and a physics-informed world model, its reliance on deterministic data collection leads to impractically slow run-times when scaled to humanoid robots. Combining this approach with PPO is a natural alternative, as PPO offers extensive parallel training infrastructure (Schwarke et al., 2025). However, our preliminary study (see Appendix A.1) highlights two advantages of SAC over PPO: its off-policy nature ensures stable convergence with limited samples, and its state-dependent stochastic actor promotes exploration in the world model, enhancing the diversity and utility of synthetic rollouts. Nevertheless, in our setting, none of these methods complete the target task within 48 hours of runtime without large-scale parallel pretraining, motivating the development of a full pretraining–finetuning pipeline using SAC as the backbone algorithm.

4.1 Large-scale Policy Pretraining

Policy Optimization. We adopt Soft Actor–Critic (SAC) (Haarnoja et al., 2018b) with an asymmetric actor–critic setup (Pinto et al., 2017): the actor $\pi_\theta(a_t|s_t)$ receives state s_t, while the critics $Q_{\psi_i}(s^p_t, a_t)$ are trained on privileged states s^p_t that include additional information. our method uses

基于模型的强化学习问题。对于微调阶段，我们寻求一个策略 $\pi_\theta \in \Pi_\theta$，以最大化模型预测的回报 $J^R_\phi(\pi_\theta)$，其定义如下：

$$J^R_\phi(\pi_\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \,\middle|\, s_0 \sim \mu,\ a_t \sim \pi_\theta(\cdot|s_t),\ (s_{t+1}, r_{t+1}) \sim \mathbb{P}_\phi(\cdot \mid s_t, a_t)\right]. \tag{1}$$

此处 $\mathbb{P}_\phi(\cdot|s_t, a_t)$ 表示由 ϕ参数化的世界模型，其近似真实环境动力学。初始状态 s_0 从真实初始分布 μ中采样，而后续状态则由学习到的模型生成。在实践中，使用随机策略 π_θ 与$\mathbb{P}_\phi$ 交互，以生成多样化的虚拟轨迹，从而提升策略优化的探索覆盖范围。当世界模型的预测足够准确时，最大化模型预测回报即等同于最大化真实环境中的回报（Janner 等人，2019；Hafner 等人，2023）。

用于机器人的物理信息世界模型将物理先验融入世界模型可提高其预测精度（Raissi 等，2019）。我们采用基于拉格朗日运动方程（Featherstone，2008）的机器人动力学物理信息公式：

$$M(q_t)\ddot{q}_t + C(q_t, \dot{q}_t) + G(q_t) = B\tau_t + J^\top F^e_t + \tau^d_t, \tag{2}$$

此处，q_t 和 $\dot{q}_t$ 分别是时间 t处的广义坐标和速度。电机扭矩 τ_t 通过矩阵 B施加到关节上。F^e_t是具有雅可比矩阵 J的外部接触力，而 τ^d_t表示耗散（寄生）扭矩。矩阵 M、 C和 G 分别为质量矩阵、科里奥利/离心项和重力项。由于 B、 M、 C和 G仅取决于机器人的形状和惯性参数，因此它们是预先已知的。因此，主要的模型不确定性在于接触项 $J^\top F^e_t$和耗散项 τ^d_t。为了对这些未知贡献进行建模，Levy 等人（2024）引入了一种残差网络来近似它们。该网络预测接触项与耗散项之和，以及下一状态预测的不确定性：

$$\tau_\phi(s_t, a_t) \approx \tau^e_t \equiv J^\top F^e_t + \tau^d_t \tag{3}$$

$$\sigma_\phi(s_t, a_t) = \log \sigma^2_t \tag{4}$$

此处，τ_ϕ 和σ_ϕ 是神经网络不同头的输出，其中 σ^2_t 表示下一状态的预测方差。这种物理信息世界模型 $\mathbb{P}_\phi$ 结合了已知的刚体动力学与学习残差，并可展开以生成更精确的轨迹用于策略优化。

4 大规模预训练与高效微调

为了使人形机器人能够在新环境中快速学习并具备高样本效率，我们首先选择最适合微调的算法，然后围绕该算法设计框架。在两个阶段使用单一算法可以统一训练目标，实现连续策略更新以缓解遗忘，并提供一个统一且易于实现的框架。虽然 SSRL（Levy 等人，2024）成功利用 SAC 和物理信息世界模型从零开始训练了四足机器人，但其对确定性数据收集的依赖导致在扩展到人形机器人时运行时间变得不切实际地缓慢。将这种方法与 PPO 结合是一种自然的替代方案，因为 PPO 提供了广泛的并行训练基础设施（Schwarke 等人，2025）。然而，我们的初步研究（见附录 A.1）突出了 SAC 相对于 PPO 的两个优势：其异策略特性确保了在有限样本下的稳定收敛，而其状态相关的随机演员促进了在世界模型中的探索，增强了合成 rollout 的多样性和实用性。尽管如此，在我们的设置中，如果没有大规模并行预训练，这些方法均无法在 48 小时的运行时间内完成目标任务，这促使我们开发了一条以 SAC 为骨干算法的完整预训练–微调流水线。

4.1 大规模策略预训练

策略优化。我们采用软演员 - 评论家算法（SAC）（Haarnoja 等人，2018b），并搭配非对称演员 - 评论家架构（Pinto 等，2017）：演员 $\pi_\theta(a_t|s_t)$ 接收状态 s_t，而评论家$Q_{\psi_i}(s^p_t, a_t)$ 则在包含额外信息的特权状态 s^p_t 上进行训练。我们的方法使用

only the robot's proprioceptive state for both the actor and the critics. We normalize each feature of s_t and s_t^p using running mean and variance (Lee et al., 2024), ensuring balanced feature scales during training. We pretrain the SAC policy in MuJoCo Playground (Zakka et al., 2025) with thousands of vectorized environments on a single GPU. Our SAC implementation is written in JAX (Bradbury et al., 2018) with fixed tensor shapes, which allows efficient operation fusion and reuse of compiled kernels. As a result, large-batch updates (high UTD) incur no additional data transfer overhead. The same design seamlessly accelerates interaction with the world model during finetuning and supports multi-GPU scaling, improving wall-clock training time. Unlike FastTD3 (Seo et al., 2025) and PQL (Li et al., 2023), we do not use per-environment mixed Gaussian noise. Instead, we rely entirely on SAC's stochastic policy for exploration, i.e., $\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$, with state-dependent variance produced by the actor and temperature α controlling entropy. We employ the `Optuna` framework (Akiba et al., 2019) for systematic hyperparameter tuning. On the Booster T1 locomotion task, we conducted approximately ten hours of hyperparameter search on a single NVIDIA RTX 4090 GPU. The results show that, after tuning, the convergence time is reduced from about seven hours to only half an hour. The training objective and hyperparameter tuning process are detailed in Appendix B.1.

Large-Batch Updates and High UTD. Raising the update-to-data (UTD) ratio reuses experience more aggressively but can amplify value-estimation bias. In our `T1LowDimJoystick` setup—SAC trained in `MuJoCo Playground` with 1,024 parallel environments, large-batch updates (batch size 1,024), and a 10^6-transition replay buffer—we observe that increasing UTD from 1 to 10 improves sample efficiency without any auxiliary stabilizers or architectural modifications. Beyond this range, gains diminish while wall-clock time increases predictably. We do not claim generality: these improvements appear tied to our massively parallel, domain-randomized regime, which likely suppresses early overfitting and curbs bias accumulation at moderate UTDs. By contrast, prior works typically couple high UTD with stabilizers such as periodic resets (D'Oro et al., 2023; Nikishin et al., 2022) or architectural regularization (ensemble/dropout critics, batch-normalized critics) (Chen et al., 2021; Hiraoka et al., 2022; Bhatt et al., 2024). We view these techniques as complementary and worth re-evaluating in large-scale parallel training for SAC to potentially push the usable UTD higher.

4.2 World Model Pretraining

Offline Pretraining from Logged Transitions Unlike MBPO or Dreamer, which update the policy and world model together online, we decouple them to improve wall-clock efficiency under massive parallelism. With thousands of parallel simulators, each step generates on the order of $\mathcal{O}(10^3)$ transitions, and training the world model online would slow the training. During SAC pretraining, we log all transitions to disk, saving records of the on-device replay buffer. After the policy converges, we train the world model entirely offline from these records. From each record, we construct $\mathcal{D} = (s_t^p, a_t, s_{t+\Delta t}^p)$. In our setting, the privileged state is $s_t^p = [q_t,\ \dot{q}_t,\ v_t^{\mathrm{b}},\ \omega_t^{\mathrm{b}},\ n_t,\ h_t]$, which includes joint angles and velocities, base linear and angular velocity in the body frame, base orientation, and body height.

World Model and Loss. We learn a residual predictor that injects external torques into a differentiable physics step. Residual predictor takes the concatenation of the privileged observation and action $\left[s_t^p,\ a_t\right]$ as input and outputs external uncertainties of torques and per-feature predictive uncertainty of the next-observation prediction:$(\tau_t^e,\ \log \sigma_t^2)$, as shown in Eq. (3). Given $[s_t^p, a_t, \tau_t^e]$, we predict the next privileged state using a fully differentiable pipeline. Concretely, we implement the world model based on Brax (Freeman et al., 2021) and perform the following steps: (a) map privileged state to Brax generalized coordinates and velocities, (b) convert actions to motor torques via a PD controller, (c) integrate the Lagrangian dynamics with semi-implicit Euler, and (d) reconstruct the next privileged state for learning and rollout. Using Brax's differentiable rigid-body primitives — $M(q)$, $C(q, \dot{q})$, and $G(q)$ — we avoid reimplementing the dynamics and keep the rollout loss pipeline end-to-end differentiable. See Appendix B.2 for details.

Compared to SSRL(Levy et al., 2024), we (i) correct the mapping from privileged state to Brax's generalized state $(q, \dot{q})$, resolving a discrepancy we identified in the publicly available SSRL implementation; and (ii) align each observation/state dimension between MuJoCo Playground and Brax (frames, quaternion form, normalization) to avoid training error;(iii) we include the base height h_t

仅机器人的本体感受状态作为演员和评论家的输入。我们利用运行均值和方差对s_t 和 s_t^p 的每个特征进行归一化（Lee 等，2024），以确保训练过程中特征缩放的平衡性。我们在 MuJoCo Playground（Zakka 等，2025）中预训练 SAC 策略，在单个 GPU 上利用数千个向量化环境并行执行。我们的 SAC 实现基于 JAX（Bradbury 等，2018）编写，采用固定张量形状，从而能够高效地进行算子融合并复用已编译的内核。因此，大批量更新（高 UTD）不会产生额外的数据传输开销。相同的设计无缝加速了微调阶段与世界模型的交互，并支持多 GPU 扩展，从而缩短挂钟时间训练耗时。与 FastTD3（Seo 等，2025）和 PQL（Li 等，2023）不同，我们不使用每环境的混合高斯分布噪声。相反，我们完全依赖 SAC 的随机策略进行探索，即 $\pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$，其方差由演员生成且与状态相关，并通过温度 α 控制熵。我们采用 Optuna 框架（Akiba 等人，2019）进行系统的超参数调优。在 Booster T1 运动任务上，我们在单块 NVIDIA RTX 4090 GPU 上进行了约十小时的超参数搜索。结果表明，经过调优后，收敛时间从约七小时缩短至仅半小时。训练目标与超参数调优过程详见附录 B.1。

大批量更新与高更新数据比。提高更新数据比（UTD）会更积极地重用经验，但也可能放大价值估计偏差。在我们的 T1LowDimJoystick 设置中——即在 MuJoCo Playground 中使用 SAC 进行训练，配备 1,024 个并行环境、大批量更新（批量大小 1,024）以及一个 10^6 步转移的回放缓冲区——我们观察到，将 UTD 从 1 提高到 10 可以在无需任何辅助稳定器或架构修改的情况下提升样本效率。超出此范围后，收益递减，而挂钟时间则按预期增加。我们不声称其具有普遍性：这些改进似乎与我们大规模并行且经过域随机化的机制有关，该机制可能抑制了早期的过拟合并遏制了中等 UTD 下的偏差积累。相比之下，先前的工作通常将高 UTD 与稳定器结合使用，例如周期性重置（D'Oro 等人，2023；Nikishin 等人，2022）或架构正则化（集成/丢弃评论家、批归一化评论家）（Chen 等人，2021；Hiraoka 等人，2022；Bhatt 等人，2024）。我们将这些技术视为互补的，并值得在 SAC 的大规模并行训练中重新评估，以 potentially 推动可用的 UTD 进一步提升。

4.2 世界模型预训练

基于记录转移的离线预训练与 MBPO 或 Dreamer 不同（后者在线联合更新策略和世界模型），我们将二者解耦，以在大规模并行下提升挂钟时间效率。拥有数千个并行仿真器时，每一步都会生成约$\mathcal{O}(10^3)$ 条转移数据，若在线训练世界模型会拖慢整体训练速度。在 SAC 预训练期间，我们将所有转移记录到磁盘，保存设备上回放缓冲区的记录。待策略收敛后，我们完全基于这些记录离线训练世界模型。从每条记录中，我们构建 $\mathcal{D} = (s_t^p, a_t, s_{t+\Delta t}^p)$。在我们的设定中，特权状态为 $s_t^p = [q_t,\ \dot{q}_t,\ v_t^{\mathrm{b}},\ \omega_t^{\mathrm{b}},\ n_t,\ h_t]$，包括关节角度和速度、体坐标系下的基座线速度和角速度、基座姿态以及机身高度。

世界模型与损失。我们学习一个残差预测器，将外部力矩注入到可微物理步骤中。该残差预测器以特权观测与动作的拼接 $\left[s_t^p\ \ a_t\right]$,作为输入，输出扭矩的外部不确定性以及下一观测预测的逐特征预测不确定性：$(\tau_t^e, \log\ \sigma_t^2)$，如公式 (3) 所示。给定 $[s_t^p, a_t, \tau_t^e]$，我们使用全可微流水线预测下一个特权状态。具体而言，我们基于 Brax（Freeman 等，2021）实现世界模型，并执行以下步骤：(a) 将特权状态映射为 Brax 的广义坐标和速度，(b) 通过 PD 控制器将动作转换为电机扭矩，(c) 使用半隐式欧拉法积分拉格朗日动力学，以及 (d) 重构下一个特权状态用于学习和推演。利用 Brax 的可微刚体基元—— $M(q)$、 $C(q, \dot{q})$ 和 $G(q)$——我们无需重新实现动力学，并保持推演损失流水线端到端可微。详见附录 B.2。

与 SSRL（Levy 等人，2024）相比，我们：(i) 修正了从特权状态到 Brax 广义状态的映射（$q,\ \dot{q}$），解决了我们在公开可用的 SSRL 实现中发现的不一致问题；(ii) 对齐了 MuJoCo Playground 和 Brax 之间的每个观测/状态维度（坐标系、四元数形式、归一化），以避免训练错误；(iii) 我们在特权状态中包含了基座高度 h_t

in the privileged state. For humanoid tasks, omitting h_t consistently caused unstable world model rollouts and non-convergent fine-tuning, whereas SSRL's quadruped configuration omitted height without issue. These adjustments improve training stability in humanoids setting.

After obtaining the next privileged state $\widehat{s}^p_{t+\Delta t}$ from the differentiable pipeline, we minimize the negative log-likelihood of a Gaussian predictive distribution. Let B denote the batch size. For prediction $\widehat{s}^p_{t+\Delta t}$, target $s^p_{t+\Delta t}$, and elementwise log-variance $\log \sigma_t^2$, the loss is

$$\mathcal{L}_\phi = \frac{1}{B}\sum_{b=1}^{B}\Big(\big(\widehat{s}^p_{b,t+\Delta t} - s^p_{b,t+\Delta t}\big)^2 \odot \exp\big(-\log\sigma^2_{b,t}\big) + \log\sigma^2_{b,t}\Big), \tag{5}$$

where $\odot$ is elementwise multiplication. Gradients backpropagate through normalization, frame transforms, the PD controller, and the Euler Step, so the residual predictor is trained end-to-end from next-state errors. Minimizing this loss encourages the world model to predict small variance for seen states and large variance for unseen states. In practice, we exploit this behavior during rollouts by sampling next states from the predicted Gaussian $\widehat{s}^p_{t+\Delta t} \sim \mathcal{N}\big(\widehat{s}^p_{t+\Delta t},\ \mathrm{diag}(\sigma_t^2)\big)$.

4.3 Policy and World Model Finetuning

Deployment & Data Collection. We deploy the pretrained policy in the target environment (Brax (Freeman et al., 2021) in our case) and collect trajectories using deterministic actions (i.e., the action mean, without exploration noise). If the robot enters an unsafe state, the episode is reset. Each episode has a maximum length of $T_{\text{ep}} = 1000$. All transitions are stored in a replay buffer, which is subsequently used to fine-tune both the world model and the policy. After collecting T_{ep} steps of data, we fine-tune the world model and policy before beginning the next iteration of data collection and training. More sophisticated pipelines, such as asynchronous data collection and learning (Luo et al., 2024), are left for future work to avoid additional complexity in this study.

World-Model Fine-Tuning. We fine-tune the pretrained world model on the collected buffer. For each epoch we independently shuffle data, roll physics inside the loss, and evaluate world model on a held-out testing data set for early stopping. We observe that training world model for multiple epochs on the same replay buffer, combined with auto-regressive training, enhances sample efficiency. The auto-regressive training objective is provided in the Appendix B.3. After World model training's done, we use the policy to explore in the world model to generate data that used to train the actor critic. Once world-model training is complete, we use the stochastic SAC actor to explore within the world model and generate trajectories.

Exploration in Physics-informed World-Model. *(a) Asymmetric Observation Generation.* Because data are collected with a deterministic policy (action mean only), the logged trajectories have limited diversity. To drive policy improvement during finetuning, we therefore explore inside the world model using a stochastic SAC policy. We keep the same asymmetric structure for actor-critic during finetuning as in pretraining: the actor consumes the proprioceptive state, while the critic uses the privileged state. Let $s^p_t \in \mathbb{R}^{d_p}$ denote the privileged state and $s_t \in \mathbb{R}^{d_s}$ the proprioceptive state, from which the action $a_t \sim \pi_\theta(\cdot|s_t)$ is sampled. The learned physics-informed world model $\mathbb{P}_\phi$ predicts the next privileged state, $\widehat{s}^p_{t+1} = \mathbb{P}_\phi(s^p_t, a_t)$ and the proprioceptive state is obtained by a fixed, simulator-consistent projection $\Pi : \mathbb{R}^{d_p} \to \mathbb{R}^{d_s}$ (same deterministic transforms used by the environment): $\widehat{s}_{t+1} = \Pi\big(\widehat{s}^p_{t+1}\big)$. *(b) Safety Reset.* For safety and numerical robustness, we apply physics-informed terminal checks and terminate a world model rollout immediately upon detecting an abnormal state—specifically, when base height, world-frame linear/angular velocities, body roll/pitch, or joint positions/velocities violate prescribed bounds. Early termination stabilizes training and prevents NaNs over long horizons. We leave richer safeguards—e.g., self-collision checks derived from link poses—as future optimization. *(c) Physics-Consistent Reward.* Since $\widehat{s}^p_t$ provides all necessary quantities (joint states, link poses, base kinematics), rewards can be computed deterministically and remain fully consistent with the underlying physics in the world model rollout. An example is provided in the Appendix B.4. In contrast, using a learned scalar predictor $\hat{r}_\psi(o^p_t, a_t)$ (Hafner et al., 2023) is unstable in deterministic data collection settings. Even small state prediction errors accumulate over long horizons, degrading policy convergence. We sample a batch of privileged states from the replay buffer as initial conditions and roll out trajectories of horizon $H_{wm} = 20$ using the world model and the stochastic policy. These synthetic trajectories are added

。对于人形机器人任务，省略 h_t consistently 导致世界模型 rollout 不稳定且微调无法收敛，而 SSRL 的四足机器人配置在省略高度时则没有问题。这些调整提高了人形机器人设置中的训练稳定性。

在从可微分流水线中获得下一个特权状态 $\widehat{s}^p_{t+\Delta t}$ 后，我们最小化高斯预测分布的负对数似然。设 B 表示批量大小。对于预测值 $\widehat{s}^p_{t+\Delta t}$、目标值 $s^p_{t+\Delta t}$ 以及逐元素对数方差 $\log\sigma_t^2$，损失函数为

$$\mathcal{L}_\phi = \frac{1}{B}\sum_{b=1}^{B}\Big(\big(\widehat{s}^p_{b,t+\Delta t} - s^p_{b,t+\Delta t}\big)^2 \odot \exp\big(-\log\sigma^2_{b,t}\big) + \log\sigma^2_{b,t}\Big), \tag{5}$$

其中 $\odot$ 表示逐元素乘法。梯度通过归一化、帧变换、PD 控制器和欧拉步进行反向传播，因此残差预测器能够从下一状态误差中实现端到端训练。最小化该损失可促使世界模型对已见状态预测较小的方差，对未见状态预测较大的方差。在实践中，我们在 rollout 期间利用这一行为，从预测的高斯分布 $\widehat{s}^p_{t+\Delta t} \sim \mathcal{N}\big(\widehat{s}^p_{t+\Delta t}\ \ \mathrm{diag}(\sigma_t^2)\big)$，中采样下一状态。

4.3 策略与世界模型微调

部署与数据收集。我们将预训练策略部署到目标环境（在本例中为 Brax (Freeman 等，2021)），并使用确定性动作（即动作均值，不含探索噪声）收集轨迹。如果机器人进入不安全状态，则重置该回合。每个回合的最大长度为 $T_{\text{ep}} = 1000$。所有转移数据均存储在回放缓冲区中，随后用于微调世界模型和策略。在收集 T_{ep}步数据后，我们在开始下一轮数据收集和训练之前，先对世界模型和策略进行微调。更复杂的流程（如异步数据收集与学习 (Luo 等人，2024)）留待未来工作，以避免本研究引入额外的复杂性。

世界模型微调。我们在收集到的回放缓冲区上对预训练世界模型进行微调。对于每一轮次，我们独立地打乱数据，在损失函数中融入物理模拟，并在留出测试数据集上评估世界模型以实施早停。我们观察到，在同一回放缓冲区上对世界模型进行多轮次训练，并结合自回归训练，能够提升样本效率。自回归训练目标详见附录 B.3。世界模型训练完成后，我们利用策略在世界模型中进行探索，生成用于训练演员 - 评论家的数据。一旦世界模型训练完成，我们便使用随机性的 SAC 策略网络在世界模型内进行探索并生成轨迹。

物理信息世界模型中的探索。*(a)* 非对称观测生成。由于数据是使用确定性策略（仅动作均值）收集的，记录的轨迹多样性有限。为了在微调期间推动策略改进，因此我们使用随机 SAC 策略在世界模型内部进行探索。我们在微调期间保持与预训练相同的 Actor-Critic 非对称结构：演员消耗本体感受状态，而评论家使用特权状态。设 $s^p_t \in \mathbb{R}^{d_p}$ 表示特权状态，$s_t \in \mathbb{R}^{d_s}$ 表示本体感受状态，从中采样动作 $a_t \sim \pi_\theta(\cdot|s_t)$。学习到的物理信息世界模型 $\mathbb{P}_\phi$ 预测下一个特权状态 $\widehat{s}^p_{t+1} = \mathbb{P}_\phi(s^p_t, a_t)$，而本体感受状态是通过固定的、模拟器一致投影 Π 获得的：$\mathbb{R}^{d_p} \to \mathbb{R}^{d_s}$（使用与环境相同的确定性变换）：$\widehat{s}_{t+1} = \Pi\big(\widehat{s}^p_{t+1}\big)$。*(b)* 安全重置。为了安全和数值鲁棒性，我们应用物理知情终止检查，并在检测到异常状态时立即终止世界模型 rollout——具体而言，当基座高度、世界坐标系线速度/角速度、机身横滚/俯仰或关节位置/速度违反规定边界时。早期终止可稳定训练并防止长时间范围内的非数值。我们将更丰富的保护措施（例如从连杆位姿导出的自碰撞检测）留作未来优化。*(c)* 物理一致奖励。由于 $\widehat{s}^p_t$ 提供了所有必要量（关节状态、连杆位姿、基座运动学），奖励可以确定性地进行计算，并在世界模型 rollout 中与底层物理完全保持一致。附录 B.4 中提供了一个示例。相比之下，在确定性数据收集设置中使用学习标量预测器 $\hat{r}_\psi(o^p_t, a_t)$（Hafner 等人，2023）是不稳定的。即使很小的状态预测误差也会在长时间范围内累积，从而降低策略收敛性。我们从回放缓冲区中采样一批特权状态作为初始条件，并使用世界模型和随机策略 rollout 时间范围为 $H_{wm} = 20$ 的轨迹。这些合成轨迹被添加

to the replay buffer to train the actor-critic via SAC. The updated policy is then deployed in the environment to collect new data, thus completing one cycle of an iterative process that continues until convergence.

5 EXPERIMENTS

Our approach performs comparably to baselines in pretraining and, importantly, enables efficient fine-tuning for humanoids in new environments with limited data. We validate on two platforms—Booster T1 and Unitree G1—and report LIFT results averaged over 8 independent runs. All experiments are executed on a single NVIDIA 4090 GPU with 32 CPU cores in a cloud setting. Evaluation uses the average undiscounted return over E episodes, $\hat{J}(\pi) = \frac{1}{E}\sum_{i=1}^{E}\sum_{t=0}^{T_{\text{ep}}} r_t$, computed without network updates. In practice, we set $E = 1024$ and $T_{\text{ep}} = 1000$. To characterize efficiency end-to-end, we compare (i) wall-clock time during pretraining (training speed) and (ii) environment steps required during fine-tuning (sample efficiency). These experiments are designed to answer three key questions: **(Q1)** Is the LIFT pretraining module comparable to established baselines? **(Q2)** Does this module better support subsequent fine-tuning than baselines? **(Q3)** Which components of our pipeline affect fine-tuning efficiency and stability?

Baseline algorithms. The baselines include: (1). **FastTD3** (Seo et al., 2025) (Model-free): An efficient variant of TD3 with parallel environments and mixed exploration noise. (2). **PPO** (Schulman et al., 2017) (Model-free): A widely used on-policy algorithm with clipped surrogate objective. We use baseline PPO in Mujoco playground. (3). **SAC** (Haarnoja et al., 2018b) (Model-free): An off-policy algorithm with stochastic actor and entropy regularization to encourage exploration. (4). **SSRL** (Levy et al., 2024) (Model-based): A physics-informed world model approach for training policies from scratch. (5). **MBPO** (Janner et al., 2019) (Model-based): Utilizes an ensemble of neural networks as the world model to improve sample efficiency of RL.

5.1 PRETRAINING EXPERIMENTS

To answer **Q1**, policy pretraining is performed in the MuJoCo Playground (Zakka et al., 2025) across six humanoid tasks, combining two terrain types (flat and rough) with three robot configurations: a low-dimensional Booster T1 (12-DoF, legs only), a full Booster T1 (23-DoF, including head, waist, arms, and legs), and the Unitree G1 (29-DoF, including waist, arms, and legs). Full details are provided in the Appendix C.2.

Pretraining results. As shown in the Appendix A.2, LIFT achieves comparable or higher evaluation returns than PPO and FastTD3 while stabilizes at its peak return faster on rough terrain environments. On flat terrain, it achieves comparable peak performance with similar wall-clock runtime. These curves indicate that the LIFT pretraining module is competitive in reward returns and convergence speed. Additionally, we also apply LIFT pretraining to whole-body tracking tasks for Unitree G1, providing preliminary evidence that the same framework extends naturally to non-locomotion skills. More details and discussion of these results are provided in Appendix A.8.

Sim-to-real with LIFT Pretrained Policy. To demonstrate the potential for zero-shot deployment, we deploy the pretrained policy for the low-dimensional Booster T1 directly on the physical robot. Zero-shot transfer to previously unseen surfaces (e.g., grass, uphill, downhill, mud, *etc*.) is shown in the Appendix A.6. This provides practical evidence that large-scale, parallel SAC pretraining can yield deployable humanoid controllers, thereby addressing **Q1**. And it also establishes a suitable starting point for subsequent fine-tuning.

5.2 FINETUNING EXPERIMENTS

Sim-to-sim Finetuning. We evaluate finetuning in Brax (Freeman et al., 2021) after large-scale policy pretraining in the MuJoCo Playground. The world model is also pretrained with data collected with pretrained policies. Because the two simulators differ in contact models and constraints, this sim-to-sim transfer provides a controlled but nontrivial test of adaptation. We transfer the pretrained policy to Brax and keep the reward design same as *Booster Gym* Wang et al. (2025) for both pretraining and finetuning. During pretraining, target linear velocities along the x-axis were uniformly sampled in $[-1, 1]$ m/s. For fine-tuning in Brax, we specify new forward-velocity targets and the

到回放缓冲区以通过 SAC 训练 Actor-Critic。然后，更新后的策略被部署到环境中以收集新数据，从而完成一个迭代过程的一个周期，该过程将持续进行直到收敛。

5 实验

我们的方法在预训练阶段表现与基线相当，更重要的是，它能够在数据有限的情况下，高效地为人形机器人在新环境中进行微调。我们在 Booster T1 和 Unitree G1 两个平台上进行了验证，并报告了 LIFT 在 8 次独立运行中的平均结果。所有实验均在云环境中的单块 NVIDIA 4090 GPU（配备 32 个 CPU 核心）上执行。评估采用 E 回合内的平均无折扣回报，$\hat{J}(\pi) = \frac{1}{E}\sum_{i=1}^{E}\sum_{t=0}^{T_{\text{ep}}} r_t$，计算过程中不进行网络更新。实践中，我们设定 $E = 1024$ 和 $T_{\text{ep}} = 1000$。为了全面表征端到端效率，我们比较了：(i) 预训练期间的挂钟时间（训练速度），以及 (ii) 微调期间所需的环境步数（样本效率）。这些实验旨在回答三个关键问题：(**Q1**)LIFT 预训练模块是否与现有基线相当？**(Q2)** 该模块是否比基线更能有效支持后续微调？**(Q3)** 我们流程中的哪些组件会影响微调的效率与稳定性？

基线算法。基线包括：(1). **FastTD3** （Seo 等，2025）（无模型）：一种高效的 TD3 变体，具有并行环境和混合探索噪声。(2). **PPO** （Schulman 等人，2017）（无模型）：一种广泛使用的同策略算法，具有截断的代理目标函数。我们在 MuJoCo Playground 中使用基线 PPO。(3). **SAC** （Haarnoja 等人，2018b）（无模型）：一种异策略算法，具有随机演员和熵正则化以鼓励探索。(4).**SSRL** （Levy 等人，2024）（基于模型）：一种物理信息世界模型方法，用于从头开始训练策略。(5). **MBPO** （Janner 等人，2019）（基于模型）：利用神经网络集成作为世界模型，以提高强化学习的样本效率。

5.1 预训练实验

为回答**Q1**，我们在 MuJoCo Playground（Zakka 等人，2025）中进行了策略预训练，涵盖六项人形机器人任务，结合两种地形类型（平坦和崎岖）与三种机器人配置：低维 Booster T1（12 自由度，仅腿部）、完整 Booster T1（23 自由度，包括头部、腰部、手臂和腿部）以及 Unitree G1（29 自由度，包括腰部、手臂和腿部）。完整详情见附录 C.2。

预训练结果。如附录 A.2 所示，在崎岖地形环境中，LIFT 实现了与 PPO 和 FastTD3 相当或更高的评估回报，并且能更快地稳定在其峰值回报。在平坦地形上，它实现了相当的峰值性能，且运行时间相似。这些曲线表明，LIFT 预训练模块在奖励回报和收敛速度方面具有竞争力。此外，我们还将 LIFT 预训练应用于 Unitree G1 的全身跟踪任务，提供了初步证据，表明该框架可自然地扩展到非移动技能。有关这些结果的更多细节和讨论见附录 A.8。

基于 LIFT 预训练策略的仿真到现实迁移。为了展示零样本部署的潜力，我们将针对低维 Booster T1 的预训练策略直接部署到物理机器人上。附录 A.6 展示了向以前未见过的表面（例如草地、上坡、下坡、泥地、等）的零样本迁移。这提供了实际证据，表明大规模并行 SAC 预训练可以产生可部署的人形控制器，从而解决了**Q1**。同时，这也为后续微调建立了合适的起点。

5.2 微调实验

仿真到仿真迁移微调。我们在 MuJoCo Playground 中进行大规模策略预训练后，评估了在 Brax（Freeman 等，2021）中的微调效果。世界模型也使用通过预训练策略收集的数据进行了预训练。由于两个仿真器在接触模型和约束方面存在差异，这种仿真到仿真迁移为适应性提供了一个受控但非平凡的测试。我们将预训练策略迁移至 Brax，并在预训练和微调过程中保持与 Booster Gym Wang 等（2025）相同的奖励设计。在预训练期间，沿 x轴的目标线速度在 $[-1, 1]$ m/s 范围内均匀采样。对于在 Brax 中的微调，我们指定了新的前向速度目标，并且

policy explores with deterministic action execution (action mean only; no stochastic sampling in the environment). We measure (i) whether policies converge within a fixed sample budget and (ii) velocity-tracking accuracy relative to the specified target. Implementation details are provided in the Appendix C.3.

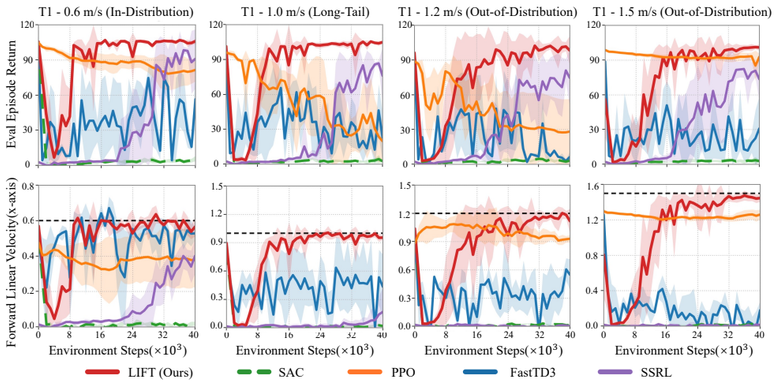


Figure 2: Results of finetuning Booster T1 robot with varying target speeds. The black dashed line represents the target velocity for each task. Results are averaged over 8 random seeds.

To address **Q2**, we design three scenarios and compare against PPO- and FastTD3-pretrained policies: **(i)In-Distribution**: targets within the pretraining range; **(ii) Long-Tail**: rare targets poorly represented during pretraining; **(iii) Out-of-Distribution**: targets outside $[-1, 1]$ m/s.

As shown in the Figure 2, LIFT consistently converges across all tasks, achieving stable walking that closely tracks the desired forward speed during the Booster T1 finetuning experiments. After finetuning, the policy demonstrates significantly reduced body oscillations and less deviation from the desired speed direction, with noticeable improvements in velocity. In contrast, SAC, trained without explicit exploration noise, quickly diverges and fails to recover, indicating rapid overfitting to the deterministically collected data. Although PPO's clipping mechanism stabilizes updates by keeping the new policy close to the previous one, its performance in our setting initially remains reasonable but then gradually degrades and ultimately collapses. This suggests that, under deterministic execution and limited data collection, PPO struggles to sustain stable policy improvement. FastTD3 exhibits strong oscillations and ultimately collapses without converging. SSRL shows signs of convergence at 0.6 m/s but fails to reach the target speed, and it does not converge at all on higher-speed tracking tasks (1.0–1.5 m/s). This highlights the difficulty of training humanoid locomotion from scratch and further motivates our pretrain–finetune design. These results demonstrate that LIFT enables robust adaptation under both in-distribution and out-of-distribution finetuning tasks, while standard baselines struggle with stability. During finetuning, policy control and data collection run at 50 Hz (one environment step = 0.02 s), so 4×10^4 steps correspond to approximately 800 s of on-robot interaction, highlighting the potential feasibility of applying LIFT on real humanoid robots which remains as future works. Additionally, we finetuned the Unitree G1 in the Brax environment, which also improved policy behavior and reward performance. The detailed results can be found in the Appendix A.7.

Real-world Finetuning. We evaluate LIFT on the Booster T1 humanoid by first pretraining a policy in the MuJoCo-Playground *T1LowDimJoystickFlatTerrain* task with most energy-related regularizations removed, keeping only an action-rate L2 penalty. This policy transfers well from MuJoCo to Brax but fails in zero-shot sim-to-real, providing a challenging starting point for real-world finetuning. Using this policy as initialization, our finetuning framework progressively improves initial unstable behavior: after collecting 80–590 s of data, the robot shows a more upright posture, smoother gait patterns, and more stable forward velocity (Fig. 3), demonstrating that LIFT can substantially strengthen a weak sim-to-real policy with only several minutes of real data. However, our current real-world implementation has two main *practical limitations*: (1) LIFT requires base-height es-

策略通过确定性动作执行进行探索（仅使用动作均值；环境中不进行随机采样）。我们测量了：(i) 策略是否在固定的样本预算内收敛，以及 (ii) 相对于指定目标的速度跟踪精度。实现细节见附录 C.3。

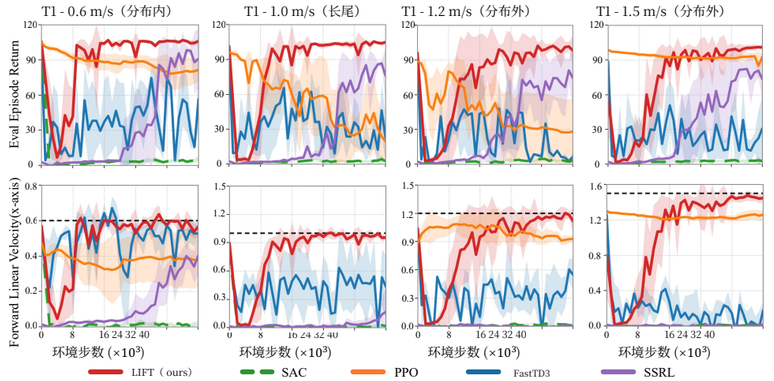


图 2：Booster T1 机器人在不同目标速度下的微调结果。黑色虚线表示每个任务的目标速度。结果为 8 个随机种子的平均值。

为解决**Q2**，我们设计了三种场景，并与基于 PPO 和 FastTD3 预训练的**(i) 分布内**策略进行比较：目标位于预训练范围内；**(ii) 长尾**：在预训练期间表征不足的罕见目标；**(iii) 分布外**：超出 $[-1, 1]$ m/s 的目标。

如图 2 所示，LIFT 在所有任务中均能稳定收敛，在 Booster T1 微调实验中实现了紧密跟踪期望前进速度的稳定行走。微调后，策略表现出显著减少的身体振荡和更小的期望速度方向偏差，速度性能有明显提升。相比之下，SAC 在没有显式探索噪声的情况下训练，迅速发散且无法恢复，表明其对确定性收集的数据快速过拟合。尽管 PPO 的截断机制通过将新策略保持在旧策略附近来稳定更新，但其在我们设定中的表现起初尚可，随后逐渐退化并最终崩溃。这表明，在确定性执行和数据收集有限的情况下，PPO 难以维持稳定的策略改进。FastTD3 表现出强烈的振荡，最终在不收敛的情况下崩溃。SSRL 在 0.6 m/s 时显示出收敛迹象，但未能达到目标速度，且在更高速度跟踪任务（1.0–1.5 m/s）上完全无法收敛。这凸显了从头训练人形机器人运动的难度，并进一步印证了我们预训练 - 微调设计的必要性。这些结果表明，LIFT 能够在分布内和分布外微调任务下实现鲁棒适应，而标准基线方法则在稳定性方面举步维艰。在微调过程中，策略控制与数据收集以 50 Hz 运行（每个环境步长 = 0.02 秒），因此 4×10^4 步对应约 800 秒的真机交互时间，突显了将 LIFT 应用于真实人形机器人的潜在可行性，这留待未来工作探索。此外，我们在 Brax 环境中对 Unitree G1 进行了微调，同样改善了策略行为和奖励性能。详细结果请参阅附录 A.7。

现实世界微调。我们在 Booster T1 人形机器人上评估 LIFT，首先在 MuJoCo-Playground 的*T1LowDimJoystickFlatTerrain* 任务中预训练一个策略，移除了大多数与能量相关的正则化项，仅保留动作率的 L2 惩罚。该策略能从 MuJoCo 良好迁移至 Brax，但在零样本仿真到现实中失败，这为现实世界微调提供了一个具有挑战性的起点。以此策略作为初始化，我们的微调框架逐步改善了初始的不稳定行为：在收集 80–590 秒数据后，机器人展现出更直立的姿态、更平滑的步态模式以及更稳定的前向速度（图 3），表明 LIFT 仅需几分钟的真实数据即可显著增强较弱的仿真到现实策略。然而，我们当前的现实世界实现存在两个主要实际限制：(1) LIFT 需要基座高度估计用于世界模型训练和奖励计算，但 Booster T1 未提供 onboard 支持，因此我们依赖 Vicon 动作捕捉系统，这限制了跟踪区域并需要人工监督。(2) 我们通过积分 IMU 加速度来估计基座线速度，这会引入漂移并可能限制策略的跟踪性能。(3) 每次微调迭代均顺序执行——包括在 50 Hz 下最多 8 秒的数据收集、世界模型更新，随后进行合成 rollout 以更新策略——导致挂钟时间长达数小时，且尽管仅使用几分钟的真实数据，仍需频繁更换电池。

timation for world-model training and rewards, but Booster T1 does not provide this onboard, so we rely on a Vicon motion-capture system, which restricts the tracking area and requires human supervision. (2) We estimate base linear velocity by integrating IMU acceleration, which introduces drift and may limit policy tracking performance. (3) Each finetuning iteration is executed sequentially—up to 8 s of data collection at 50 Hz, world model update, then synthetic rollouts for policy updates—leading to multi-hour wall-clock time and frequent battery swaps despite using only minutes of real data. We view these as engineering rather than conceptual constraints, and expect that adopting an asynchronous pipeline similar to SERL (Luo et al., 2024), together with camera-based height and velocity estimation onboard, would make repeated real-world finetuning substantially more practical. More results are provided in Appendix A.3.

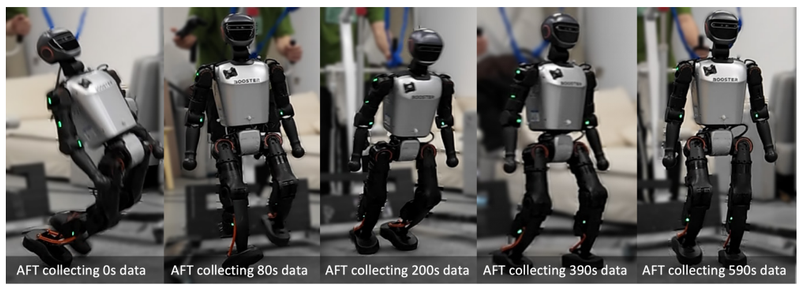


Figure 3: Real-world finetuning progression on the Booster T1 humanoid. A video demonstration is available on our project website.

5.3 Ablation Study

To answer **Q3** and identify which components of LIFT affect fine-tuning efficiency and stability, ablations over three aspects are conducted: (i) *pretraining stages*—large-scale SAC pretraining and world-model (WM) pretraining; (ii) *world-model choice*—our physics-informed model versus an MBPO-style ensemble; and (iii) *key hyperparameters*—UTD ratio, replay-buffer size, batch size (pretraining), entropy coefficient α and autoregressive loss horizon (fine-tuning).

Effect of Pretraining. As shown in the Figure 4, we ablate WM and SAC pretraining. With both pretraining stages, LIFT converges within 4×10^4 environment steps and successfully tracks the target speed. Removing WM pretraining still allows the method to converge eventually, but noticeably slows down training, indicating that WM pretraining improves sample efficiency. Removing both pretraining stages reduces the LIFT to SSRL Levy et al. (2024), which mostly learns to stand in place with near-zero forward velocity. Thus, large-scale SAC pretraining is essential to avoid poor local minima, and WM pretraining further improves finetuning efficiency and stability.

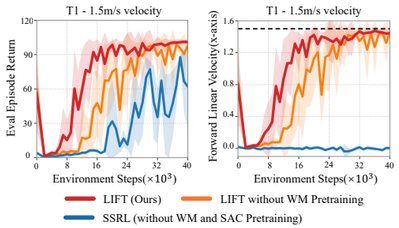


Figure 4: Ablation of the pretraining on Booster T1 (target forward speed $= 1.5$ m/s). Results are averaged over 8 random seeds.

Physics-informed vs. Non-physics-informed World Models We pretrain MBPO's ensemble world model (ensemble size $= 5$, elite size $= 3$) on the same dataset and finetune with identical hyperparameters to LIFT; the only difference is the choice of world model. MBPO fails to converge: episode return remain near zero (training curves as shown in Figure 5). On the test set its mean squared error (MSE) of world model is substantially worse than LIFT's. During model-based rollouts, a stochastic policy frequently produces actions that lie outside the distribution that has been seen in world-model training. These out-of-distribution actions induce physically implausible predictions (e.g., body height) for MBPO, which cause the critic loss to explode and inhibit policy improvement. This behavior likely stems from the purely neural network's limited ability to

我们将这些视为工程性而非概念性约束，并预期采用类似 SERL（Luo 等人，2024）的异步流水线，结合 onboard 基于摄像头的高度与速度估计，将使重复的现实世界微调变得 substantially 更具可行性。更多结果见附录 A.3。

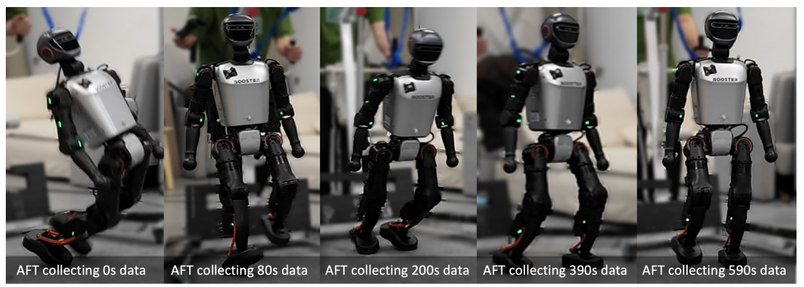


图 3：Booster T1 人形机器人在真实世界中的微调进展。视频演示可在我们的项目网站上查看。

5.3 消融研究

为回答**Q3**并确定 LIFT 的哪些组件影响微调效率和稳定性，我们对三个方面进行了消融实验：(i) 预训练阶段——大规模 SAC 预训练和世界模型（WM）预训练；(ii) 世界模型选择——我们的物理信息模型与 MBPO 风格集成；以及 (iii) 关键超参数——UTD 比率、回放缓冲区大小、批量大小（预训练）、熵系数 α 和自回归损失范围（微调）。

预训练的效果。如图 4 所示，我们对世界模型（WM）和 SAC 预训练进行了消融实验。在两个预训练阶段均存在的情况下，LIFT 能在 4×10^4 环境步数内收敛，并成功跟踪目标速度。移除世界模型预训练后，方法最终仍能收敛，但训练速度明显变慢，这表明世界模型预训练提升了样本效率。若同时移除两个预训练阶段，LIFT 则退化为 SSRL（Levy 等人，2024），其大多仅学会原地站立，前向速度接近于零。因此，大规模 SAC 预训练对于避免陷入不良局部极小值至关重要，而世界模型预训练则进一步提升了微调效率与稳定性。

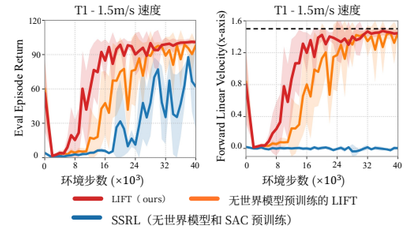


图 4：Booster T1 上预训练的消融实验（目标前进速度 $= 1.5$m/s）。结果取 8 个随机种子的平均值。

物理信息与非物理信息世界模型我们在相同数据集上预训练 MBPO 的集成世界模型（集成规模 $= 5$，精英规模 $= 3$），并使用与 LIFT 相同的超参数进行微调；唯一的区别在于世界模型的选择。MBPO 无法收敛：回合回报始终接近于零（训练曲线如图 5 所示）。在测试集上，其世界模型的均方误差（MSE）显著差于 LIFT。在基于模型的 rollout 过程中，随机策略经常生成在世界模型训练中未曾见过的分布外动作。这些分布外动作会导致 MBPO 产生物理上不可信的预测（例如机身高度），从而引发评论家损失爆炸并阻碍策略改进。这种行为可能源于纯神经网络泛化能力的局限性，

generalize. By contrast, LIFT's physics-informed world model supplies strong inductive priors that improve generalization under limited data and produce stable, learnable rollouts, enabling successful finetuning.

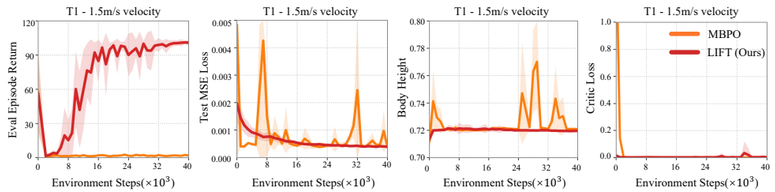


Figure 5: Ablation of Physics informed World Model on Booster T1 (target speed $=$ 1.5 m/s). Results are averaged over 8 random seeds.

Effect of Hyperparameters. During pretraining, we find that the UTD ratio, buffer size, and batch size strongly influence convergence speed and performance. UTD $= 1$ converges very slowly, while increasing it to 5 speeds up learning. Larger buffer and batch sizes further accelerate convergence, though excessively large buffers increase GPU memory usage. Beyond certain thresholds, larger batch sizes or higher UTD provide little additional benefit. In finetuning, the entropy coefficient α and autoregressive horizon length are critical. Excessively large α promotes exploration into high-uncertainty regions of the world model, destabilizing learning. Using α from pretraining works reasonably, but smaller values achieve more stable convergence. A loss horizon length of 1 sometimes fails to reach the target velocity, while lengths of 2 or 4 consistently ensure stable learning, indicating that multi-step autoregressive prediction improves world-model training and policy stability. The training curves are provided in the Appendix A.4.

6 Conclusion and Discussion

In this work, we propose LIFT, a pretraining–finetuning framework for humanoid control that bridges large-scale simulation and data-efficient adaptation. By leveraging massively parallel environments with SAC, LIFT enables fast, robust pretraining and zero-shot sim-to-real transfer. The pretrained policy then guides model-based fine-tuning, while the same pretraining data bootstrap a physics-informed world model to improve sample efficiency. During fine-tuning, deterministic action execution is combined with stochastic exploration inside the world model, providing stable learning under limited data. Experimental results highlight a practical path toward continuous, efficient humanoid learning.

Despite these results, there still exist limitations in our current work that inspire several future directions: (1) *Safety management during real-world finetuning:* collecting deterministic trajectories in the real world still carries inherent risks due to model errors in the actor network. We enforce strict termination conditions, aligned with those in simulation, to halt episodes when key physical quantities exceed predefined thresholds. Human operators also remotely terminate any episode exhibiting unsafe behaviors (e.g., drifting toward obstacles). Upon termination, the policy is halted and the Booster T1 is switched to damping mode to restrict further motion. The robot is then guided back to the starting area using its default walking controller, followed by a standing controller to reset it to a consistent initial configuration. This procedure maintains stable initial states across episodes and mitigates distribution mismatch during data collection. Future work may incorporate more automated safety mechanisms—such as robot-assisted resetting (Hu et al., 2025), uncertainty-aware exploration (An et al., 2021; Yu et al., 2020), or recovery policies with safety switches (Thananjeyan et al., 2021; Smith et al., 2022a)—to further improve safety and reduce human intervention during real-world humanoid finetuning; (2) *high-dimensional external sensor and visual inputs:* LIFT currently operates exclusively on proprioceptive observations and does not incorporate camera or other high-dimensional sensory inputs, in contrast to vision-based frameworks such as DreamerV3 (Hafner et al., 2023). Scaling LIFT to tasks with vision-centric objectives—such as dexterous manipulation or object-centric control—will likely require latent world models capable of capturing dynamics beyond the robot's body state, which we view as a promising direction for future extensions.

相比之下，LIFT 的物理信息世界模型提供了强大的归纳先验，提升了数据受限情况下的泛化能力，并生成了稳定且可学习的 rollout，从而实现了成功的微调。

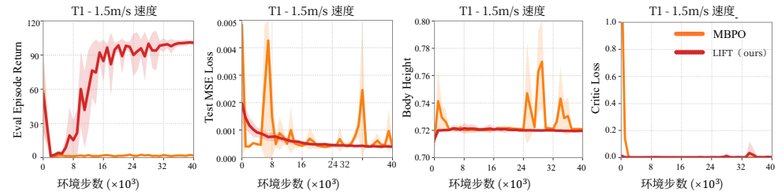


图 5：物理信息世界模型在 Booster T1 上的消融实验（目标速度 $=$ 1.5m/s）。结果为 8 个随机种子的平均值。

超参数的影响。在预训练期间，我们发现 UTD 比率、缓冲区大小和批量大小对收敛速度和性能有显著影响。UTD $= 1$ 收敛非常缓慢，而将其增加到 5 可加速学习。更大的缓冲区和批量大小能进一步加快收敛，但过大的缓冲区会增加 GPU 内存使用量。超过某些阈值后，更大的批量大小或更高的 UTD 带来的额外收益微乎其微。在微调阶段，熵系数 α 和自回归视界长度至关重要。过大的 α 会促使探索进入世界模型的高不确定性区域，从而 destabilizing 学习。使用预训练中的 α 效果尚可，但较小的值可实现更稳定的收敛。损失视界长度为 1 时有时无法达到目标速度，而长度为 2 或 4 则能始终确保稳定学习，这表明多步自回归预测提升了世界模型训练和策略稳定性。训练曲线见附录 A.4。

6 结论与讨论

在本工作中，我们提出了 LIFT，这是一种用于人形控制的预训练–微调框架，旨在弥合大规模仿真与数据高效适应之间的差距。通过利用大规模并行环境与 SAC，LIFT 实现了快速、鲁棒的预训练以及零样本仿真到现实迁移。预训练策略随后指导基于模型的微调，而相同的预训练数据则用于引导构建物理信息世界模型，以提升样本效率。在微调过程中，确定性动作执行与世界模型内的随机探索相结合，从而在有限数据下实现稳定学习。实验结果凸显了一条通往连续、高效人形机器人学习的实用路径。

尽管取得了这些成果，我们当前的工作仍存在局限性，这启发了几个未来的研究方向：(1)真实世界微调期间的安全管理：由于执行器网络中的模型误差，在真实世界中收集确定性轨迹仍然带有固有风险。我们执行严格的终止条件，与仿真中的条件保持一致，当关键物理量超过预定义阈值时停止回合。人类操作员也会远程终止任何表现出不安全行为（例如漂向障碍物）的回合。一旦终止，策略即被 halted，Booster T1 切换至阻尼模式以限制进一步运动。随后，机器人利用其默认行走控制器被引导回起始区域，再通过站立控制器将其重置为一致的初始构型。该过程确保了各回合间初始状态的稳定性，并减轻了数据收集过程中的分布不匹配。未来的工作可以纳入更多自动化安全机制——例如机器人辅助重置（Hu 等，2025）、不确定性感知探索（An 等，2021；Yu 等，2020），或带有安全开关的恢复策略（Thananjeyan 等，2021；Smith 等，2022a）——以进一步提高安全性并减少真实世界人形机器人微调期间的人工干预；(2) 高维外部传感器和视觉输入：LIFT 目前仅基于本体感知观测运行，未整合相机或其他高维感官输入，这与 DreamerV3（Hafner 等人，2023）等基于视觉的框架形成对比。将 LIFT 扩展至具有视觉中心目标的任务——例如灵巧操作或对象中心控制——可能需要能够捕捉超越机器人身体状态动力学的隐变量世界模型，我们认为这是未来扩展的一个有前景的方向。

ACKNOWLEDGEMENT

This work was supported in part by the National Natural Science Foundation of China (No. 62403064, 62403063) and Shenzhen Science and Technology Program (No. ZDCY20250901094531003). We thank the engineering team from Booster Robotics for technical support.

Reproducibility Statement. We train all LIFT agents on a single NVIDIA RTX 4090 GPU. Our experiments use the MuJoCo Playground[1] and Brax[2] simulators. The source code and full results are available on our project website: `https://lift-humanoid.github.io/`. Our baseline implementations are adapted from the following open-source repositories: PPO[3], FastTD3[4], MBPO, and SSRL[5]. Detailed hyperparameters, additional experiments, and environment configurations are provided in Appendix A, B and C.

REFERENCES

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631, 2019.

Eitan Altman. *Constrained Markov decision processes: stochastic modeling*. Routledge, 1999.

Gaon An, Seungyong Moon, Jang-Hyun Kim, and Hyun Oh Song. Uncertainty-based offline reinforcement learning with diversified q-ensemble. *Advances in neural information processing systems*, 34:7436–7447, 2021.

Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pp. 1577–1594. PMLR, 2023.

Aditya Bhatt, Daniel Palenicek, Boris Belousov, Max Argus, Artemij Amiranashvili, Thomas Brox, and Jan Peters. Crossq: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=PczQtTsTIX`.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/jax-ml/jax`.

Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations (ICLR)*, 2021. URL `https://openreview.net/forum?id=AY8zfZm0tDd`.

Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, and Shirley Ho. Lagrangian neural networks. *arXiv preprint arXiv:2003.04630*, 2020.

Pierluca D'Oro, Max Schwarzer, Evgenii Nikishin, Pierre-Luc Bacon, Marc G. Bellemare, and Aaron Courville. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *International Conference on Learning Representations (ICLR)*, 2023. URL `https://openreview.net/forum?id=OpC-9aBBVJe`.

Roy Featherstone. *Rigid body dynamics algorithms*. Springer, 2008.

C Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax–a differentiable physics engine for large scale rigid body simulation. *arXiv preprint arXiv:2106.13281*, 2021.

[1] `https://github.com/google-deepmind/mujoco_playground`
[2] `https://github.com/google/brax`
[3] `https://github.com/google/brax`
[4] `https://github.com/younggyoseo/FastTD3`
[5] `https://github.com/CLeARoboticsLab/ssrl`

致谢

本工作部分得到了国家自然科学基金委员会（编号：62403064、62403063）和深圳市科技计划（编号：ZDCY20250901094531003）的支持。我们感谢 Booster Robotics 工程团队提供的技术支持。

可复现性声明。我们在单块 NVIDIA RTX 4090 GPU 上训练所有 LIFT 智能体。我们的实验使用了 MuJoCo Playground[1] 和 Brax[2] 仿真器。源代码和完整结果可在我们的项目网站获取：https://lift-humanoid.github.io/。我们的基线实现改编自以下开源仓库：PPO[3]，、FastTD3[4]，、MBPO 和 SSRL[5]。详细的超参数、额外实验及环境配置见附录 A、B 和 C。

参考文献

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 2623–2631, 2019.

Eitan Altman. *Constrained Markov decision processes: stochastic modeling*. Routledge, 1999.

Gaon An, Seungyong Moon, Jang-Hyun Kim, and Hyun Oh Song. Uncertainty-based offline reinforcement learning with diversified q-ensemble. *Advances in neural information processing systems*, 34:7436–7447, 2021.

Philip J Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, pp. 1577–1594. PMLR, 2023.

Aditya Bhatt, Daniel Palenicek, Boris Belousov, Max Argus, Artemij Amiranashvili, Thomas Brox, and Jan Peters. Crossq: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International Conference on Learning Representations (ICLR)*, 2024. URL `https://openreview.net/forum?id=PczQtTsTIX`.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/jax-ml/jax`.

Xinyue Chen, Che Wang, Zijian Zhou, and Keith W. Ross. Randomized ensembled double q-learning: Learning fast without a model. In *International Conference on Learning Representations (ICLR)*, 2021. URL `https://openreview.net/forum?id=AY8zfZm0tDd`.

Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, and Shirley Ho. Lagrangian neural networks. *arXiv preprint arXiv:2003.04630*, 2020.

Pierluca D'Oro, Max Schwarzer, Evgenii Nikishin, Pierre-Luc Bacon, Marc G. Bellemare, and Aaron Courville. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *International Conference on Learning Representations (ICLR)*, 2023. URL `https://openreview.net/forum?id=OpC-9aBBVJe`.

Roy Featherstone. *Rigid body dynamics algorithms*. Springer, 2008.

C Daniel Freeman, Erik Frey, Anton Raichuk, Sertan Girgin, Igor Mordatch, and Olivier Bachem. Brax–a differentiable physics engine for large scale rigid body simulation. *arXiv preprint arXiv:2106.13281*, 2021.

[1] https://github.com/google-deepmind/mujoco_游乐场[2]
https://github.com/google/brax[3]https://github.co
m/google/brax[4]https://github.com/younggyoseo/FastTD3[5]
https://github.com/CLeARoboticsLab/ssrl

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *CoRR*, abs/1802.09477, 2018. URL http://arxiv.org/abs/1802.09477.

Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. *Advances in neural information processing systems*, 32, 2019.

Xinyang Gu, Yen-Jen Wang, and Jianyu Chen. Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer. *arXiv preprint arXiv:2404.05695*, 2024.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3), 2018.

Sehoon Ha, Peng Xu, Zhenyu Tan, Sergey Levine, and Jie Tan. Learning to walk in the real world with minimal human effort. *arXiv preprint arXiv:2002.08550*, 2020.

Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning. *arXiv preprint arXiv:1812.11103*, 2018a.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. Pmlr, 2018b.

Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018c.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Nicklas Hansen, Jyothir SV, Vlad Sobal, Yann LeCun, Xiaolong Wang, and Hao Su. Hierarchical world models as visual whole-body humanoid controllers. *arXiv preprint arXiv:2405.18418*, 2024.

Tairan He, Jiawei Gao, Wenli Xiao, Yuanhang Zhang, Zi Wang, Jiashun Wang, Zhengyi Luo, Guanqi He, Nikhil Sobanbab, Chaoyi Pan, et al. Asap: Aligning simulation and real-world physics for learning agile humanoid whole-body skills. *arXiv preprint arXiv:2502.01143*, 2025.

Takuya Hiraoka, Takahisa Imagawa, Taisei Hashimoto, Takashi Onishi, and Yoshimasa Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=xCVJMsPv3RT.

Kaizhe Hu, Haochen Shi, Yao He, Weizhuo Wang, C Karen Liu, and Shuran Song. Robot trains robot: Automatic real-world policy adaptation and learning for humanoids. *arXiv preprint arXiv:2508.12252*, 2025.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.

Hojoon Lee, Dongyoon Hwang, Donghu Kim, Hyunseung Kim, Jun Jet Tai, Kaushik Subramanian, Peter R Wurman, Jaegul Choo, Peter Stone, and Takuma Seno. Simba: Simplicity bias for scaling up parameters in deep reinforcement learning. *arXiv preprint arXiv:2410.09754*, 2024.

Jacob Levy, Tyler Westenbroek, and David Fridovich-Keil. Learning to walk from three minutes of real-world data with semi-structured dynamics models. *arXiv preprint arXiv:2410.09163*, 2024.

Zechu Li, Tao Chen, Zhang-Wei Hong, Anurag Ajay, and Pulkit Agrawal. Parallel q-learning: Scaling off-policy reinforcement learning under massively parallel simulation. In *International Conference on Machine Learning*, pp. 19440–19459. PMLR, 2023.

Qiayuan Liao, Takara E Truong, Xiaoyu Huang, Guy Tevet, Koushil Sreenath, and C Karen Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.

Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *CoRR*, abs/1802.09477, 2018. URL http://arxiv.org/abs/1802.09477.

Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. *Advances in neural information processing systems*, 32, 2019.

Xinyang Gu, Yen-Jen Wang, and Jianyu Chen. Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer. *arXiv preprint arXiv:2404.05695*, 2024.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3), 2018.

Sehoon Ha, Peng Xu, Zhenyu Tan, Sergey Levine, and Jie Tan. Learning to walk in the real world with minimal human effort. *arXiv preprint arXiv:2002.08550*, 2020.

Tuomas Haarnoja, Sehoon Ha, Aurick Zhou, Jie Tan, George Tucker, and Sergey Levine. Learning to walk via deep reinforcement learning. *arXiv preprint arXiv:1812.11103*, 2018a.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pp. 1861–1870. Pmlr, 2018b.

Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, et al. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018c.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Nicklas Hansen, Jyothir SV, Vlad Sobal, Yann LeCun, Xiaolong Wang, and Hao Su. Hierarchical world models as visual whole-body humanoid controllers. *arXiv preprint arXiv:2405.18418*, 2024.

Tairan He, Jiawei Gao, Wenli Xiao, Yuanhang Zhang, Zi Wang, Jiashun Wang, Zhengyi Luo, Guanqi He, Nikhil Sobanbab, Chaoyi Pan, et al. Asap: Aligning simulation and real-world physics for learning agile humanoid whole-body skills. *arXiv preprint arXiv:2502.01143*, 2025.

Takuya Hiraoka, Takahisa Imagawa, Taisei Hashimoto, Takashi Onishi, and Yoshimasa Tsuruoka. Dropout q-functions for doubly efficient reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=xCVJMsPv3RT.

Kaizhe Hu, Haochen Shi, Yao He, Weizhuo Wang, C Karen Liu, and Shuran Song. Robot trains robot: Automatic real-world policy adaptation and learning for humanoids. *arXiv preprint arXiv:2508.12252*, 2025.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.

Hojoon Lee, Dongyoon Hwang, Donghu Kim, Hyunseung Kim, Jun Jet Tai, Kaushik Subramanian, Peter R Wurman, Jaegul Choo, Peter Stone, and Takuma Seno. Simba: Simplicity bias for scaling up parameters in deep reinforcement learning. *arXiv preprint arXiv:2410.09754*, 2024.

Jacob Levy, Tyler Westenbroek, and David Fridovich-Keil. Learning to walk from three minutes of real-world data with semi-structured dynamics models. *arXiv preprint arXiv:2410.09163*, 2024.

Zechu Li, Tao Chen, Zhang-Wei Hong, Anurag Ajay, and Pulkit Agrawal. Parallel q-learning: Scaling off-policy reinforcement learning under massively parallel simulation. In *International Conference on Machine Learning*, pp. 19440–19459. PMLR, 2023.

Qiayuan Liao, Takara E Truong, Xiaoyu Huang, Guy Tevet, Koushil Sreenath, and C Karen Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16961–16969. IEEE, 2024.

Michael Lutter, Christian Ritter, and Jan Peters. Deep lagrangian networks: Using physics as model prior for deep learning. *arXiv preprint arXiv:1907.04490*, 2019.

Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

Evgenii Nikishin, Max Schwarzer, Pierluca D'Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pp. 16828–16847. PMLR, 2022. URL https://proceedings.mlr.press/v162/nikishin22a.html.

Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. *arXiv preprint*, arXiv:1710.06542, 2017. URL https://arxiv.org/abs/1710.06542.

Antonin Raffin. Getting sac to work on a massive parallel simulator: An rl journey with off-policy algorithms. *araffin.github.io*, Feb 2025. URL https://araffin.github.io/post/sac-massive-sim/.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning*, pp. 91–100. PMLR, 2022.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Clemens Schwarke, Mayank Mittal, Nikita Rudin, David Hoeller, and Marco Hutter. Rsl-rl: A learning library for robotics research. *arXiv preprint arXiv:2509.10771*, 2025.

Younggyo Seo, Carmelo Sferrazza, Haoran Geng, Michal Nauman, Zhao-Heng Yin, and Pieter Abbeel. Fasttd3: Simple, fast, and capable reinforcement learning for humanoid control. *arXiv preprint arXiv:2505.22642*, 2025.

Laura Smith, J Chase Kew, Xue Bin Peng, Sehoon Ha, Jie Tan, and Sergey Levine. Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *2022 international conference on robotics and automation (ICRA)*, pp. 1593–1599. IEEE, 2022a.

Laura Smith, Ilya Kostrikov, and Sergey Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860*, 2022b.

Wandong Sun, Long Chen, Yongbo Su, Baoshi Cao, Yang Liu, and Zongwu Xie. Learning humanoid locomotion with world model reconstruction. *arXiv preprint arXiv:2502.16230*, 2025.

Brijen Thananjeyan, Ashwin Balakrishna, Suraj Nair, Michael Luo, Krishnan Srinivasan, Minho Hwang, Joseph E Gonzalez, Julian Ibarz, Chelsea Finn, and Ken Goldberg. Recovery rl: Safe reinforcement learning with learned recovery zones. *IEEE Robotics and Automation Letters*, 6 (3):4915–4922, 2021.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Jianlan Luo, Zheyuan Hu, Charles Xu, You Liang Tan, Jacob Berg, Archit Sharma, Stefan Schaal, Chelsea Finn, Abhishek Gupta, and Sergey Levine. Serl: A software suite for sample-efficient robotic reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 16961–16969. IEEE, 2024.

Michael Lutter, Christian Ritter, and Jan Peters. Deep lagrangian networks: Using physics as model prior for deep learning. *arXiv preprint arXiv:1907.04490*, 2019.

Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, et al. Isaac gym: High performance gpu-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

Evgenii Nikishin, Max Schwarzer, Pierluca D'Oro, Pierre-Luc Bacon, and Aaron Courville. The primacy bias in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pp. 16828–16847. PMLR, 2022. URL https://proceedings.mlr.press/v162/nikishin22a.html.

Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. *arXiv preprint*, arXiv:1710.06542, 2017. URL https://arxiv.org/abs/1710.06542.

Antonin Raffin. Getting sac to work on a massive parallel simulator: An rl journey with off-policy algorithms. *araffin.github.io*, Feb 2025. URL https://araffin.github.io/post/sac-massive-sim/.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.

Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Conference on Robot Learning*, pp. 91–100. PMLR, 2022.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Clemens Schwarke, Mayank Mittal, Nikita Rudin, David Hoeller, and Marco Hutter. Rsl-rl: A learning library for robotics research. *arXiv preprint arXiv:2509.10771*, 2025.

Younggyo Seo, Carmelo Sferrazza, Haoran Geng, Michal Nauman, Zhao-Heng Yin, and Pieter Abbeel. Fasttd3: Simple, fast, and capable reinforcement learning for humanoid control. *arXiv preprint arXiv:2505.22642*, 2025.

Laura Smith, J Chase Kew, Xue Bin Peng, Sehoon Ha, Jie Tan, and Sergey Levine. Legged robots that keep on learning: Fine-tuning locomotion policies in the real world. In *2022 international conference on robotics and automation (ICRA)*, pp. 1593–1599. IEEE, 2022a.

Laura Smith, Ilya Kostrikov, and Sergey Levine. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860*, 2022b.

Wandong Sun, Long Chen, Yongbo Su, Baoshi Cao, Yang Liu, and Zongwu Xie. Learning humanoid locomotion with world model reconstruction. *arXiv preprint arXiv:2502.16230*, 2025.

Brijen Thananjeyan, Ashwin Balakrishna, Suraj Nair, Michael Luo, Krishnan Srinivasan, Minho Hwang, Joseph E Gonzalez, Julian Ibarz, Chelsea Finn, and Ken Goldberg. Recovery rl: Safe reinforcement learning with learned recovery zones. *IEEE Robotics and Automation Letters*, 6 (3):4915–4922, 2021.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 23–30. IEEE, 2017.

Yushi Wang, Penghui Chen, Xinyu Han, Feng Wu, and Mingguo Zhao. Booster gym: An end-to-end reinforcement learning framework for humanoid robot locomotion. *arXiv preprint arXiv:2506.15132*, 2025.

Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*, pp. 2226–2240. PMLR, 2023.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. *Advances in Neural Information Processing Systems*, 33:14129–14142, 2020.

Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A Kahrs, et al. Mujoco playground. *arXiv preprint arXiv:2502.08844*, 2025.

Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pp. 23–30. IEEE, 2017.

Yushi Wang, Penghui Chen, Xinyu Han, Feng Wu, and Mingguo Zhao. Booster gym: An end-to-end reinforcement learning framework for humanoid robot locomotion. *arXiv preprint arXiv:2506.15132*, 2025.

Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*, pp. 2226–2240. PMLR, 2023.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. *Advances in Neural Information Processing Systems*, 33:14129–14142, 2020.

Kevin Zakka, Baruch Tabanpour, Qiayuan Liao, Mustafa Haiderbhai, Samuel Holt, Jing Yuan Luo, Arthur Allshire, Erik Frey, Koushil Sreenath, Lueder A Kahrs, et al. Mujoco playground. *arXiv preprint arXiv:2502.08844*, 2025.

A Additional Experiments

A.1 Preliminary Study

We consider the practical case where the pretrained policy entirely fails in the deployment stage, finetuning must effectively learn a new policy from scratch, **with data collection constrained to deterministic policy**. To study this setting, we design a simple experiment using the BoosterT1 robot, which has 12 lower-body degrees of freedom. The reward is composed of three terms commonly used in humanoid locomotion: a swing-leg reference, body-height tracking, and linear velocity tracking. The policy is trained to achieve a target forward velocity of 0.2 m/s. The implementation details can be found in Appendix C.1.

As shown in Figure 6, we compare PPO and SAC within the SSRL (Levy et al., 2024) framework. The default setting uses SAC, which leverages a state-dependent stochastic actor to explore in the world model and improve policy performance. For comparison, we replace SAC with PPO (implemented in RSL-RL (Schwarke et al., 2025)) while keeping all other components fixed, tuning only the hyperparameters. We observe that the most critical factor for PPO performance is the initial action standard deviation (std), which strongly influences the distribution of states explored in the world model. A larger std makes training difficult to converge and tends to drive the policy into regions where the world model is poorly trained, while a smaller std leads to under-exploration and unstable convergence, with some seeds failing to learn at all. To mitigate this, we replace the PPO actor with the SAC actor that outputs both mean and std of the action distribution. This modification greatly improves stability, with all 8 seeds converging, but still requires two to three times more samples than SAC. Meanwhile, we run these experiments for 48 hours on a single Brax (Freeman et al., 2021) simulation. None of the methods converge to the target velocity within this time, highlighting the need for large-scale pretraining to reduce wall-clock time and avoid convergence to local minima.

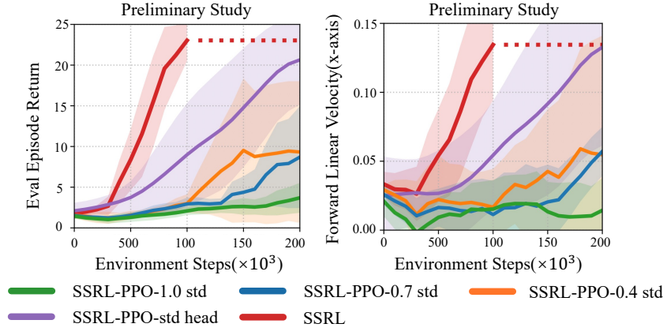


Figure 6: Comparison of PPO and SAC in a preliminary fine-tuning experiment with the BoosterT1 robot, run for 48 hours on a single Brax simulation. The left figure shows the evaluation episode return, and the right figure shows the body's linear velocity along the x-axis. Curves represent the average over 8 random seeds, with evaluation performed in 128 parallel environments. PPO performance is highly sensitive to the initial action standard deviation (std). Replacing the PPO actor with a SAC-style actor that outputs both the mean and standard deviation improves stability, although SAC still requires fewer samples to achieve the same performance.

A 附加实验

A.1 初步研究

我们考虑一种实际场景，其中预训练策略在部署阶段完全失效，微调必须从头开始有效学习新策略，**且数据收集受限于确定性策略**。为了研究这一设定，我们设计了一个使用 BoosterT1 机器人的简单实验，该机器人具有 12 个下肢自由度。奖励由人形机器人运动中常用的三项组成：摆动腿参考、身体高度跟踪和线速度跟踪。策略被训练以实现 0.2 m/s 的目标前进速度。实现细节见附录 C.1。

如图 6 所示，我们在 SSRL（Levy 等人，2024）框架内比较了 PPO 和 SAC。默认设置使用 SAC，它利用状态依赖的随机策略网络在世界模型中进行探索并提升策略性能。为了对比，我们将 SAC 替换为 PPO（在 RSL-RL（Schwarke 等人，2025）中实现），同时保持所有其他组件不变，仅调整超参数。我们观察到，影响 PPO 性能的最关键因素是初始动作标准差（std），它强烈影响了在世界模型中探索的状态分布。较大的 std 会导致训练难以收敛，并倾向于将策略驱动到世界模型训练不足的区域；而较小的 std 则会导致探索不足和收敛不稳定，部分随机种子甚至完全无法学习。为缓解这一问题，我们将 PPO 演员替换为能输出动作分布均值和标准差的 SAC 策略网络。这一改进显著提升了稳定性，使全部 8 个随机种子均能收敛，但仍需要比 SAC 多两到三倍的样本量。与此同时，我们在单个 Brax（Freeman 等，2021）仿真上运行这些实验长达 48 小时。在此期间，没有任何方法能够收敛到目标速度，这凸显了大规模预训练的必要性，以减少挂钟时间并避免陷入局部极小值。

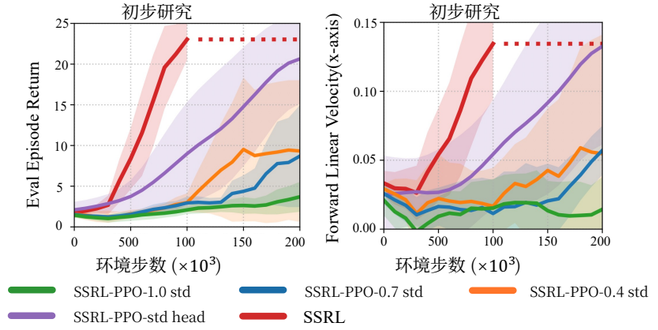


图 6：BoosterT1 机器人初步微调实验中 PPO 与 SAC 的对比，在单个 Brax 仿真中运行 48 小时。左图显示评估回合回报，右图显示机体沿 x轴的线速度。曲线代表 8 个随机种子的平均值，评估在 128 个并行环境中进行。PPO 性能对初始动作标准差（std）高度敏感。将 PPO 演员替换为同时输出均值和标准差的 SAC 风格策略网络可提高稳定性，尽管 SAC 仍需要更少的样本即可达到相同性能。

A.2 PRETRAINING EXPERIMENTS

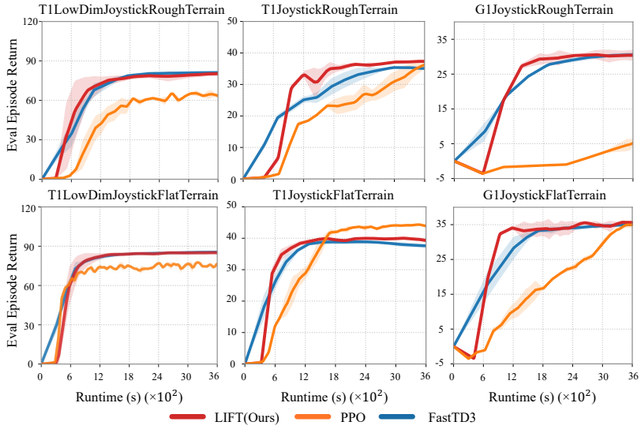


Figure 7: Pretraining performance comparison of LIFT (red), PPO (orange), and FastTD3 (blue) across six humanoid tasks (top row: rough terrain for the three robot configurations; bottom row: flat terrain for the three robot configurations). Results show the mean over 8 random seeds.

A.3 FINETUNING EXPERIMENTS IN THE REAL-WORLD

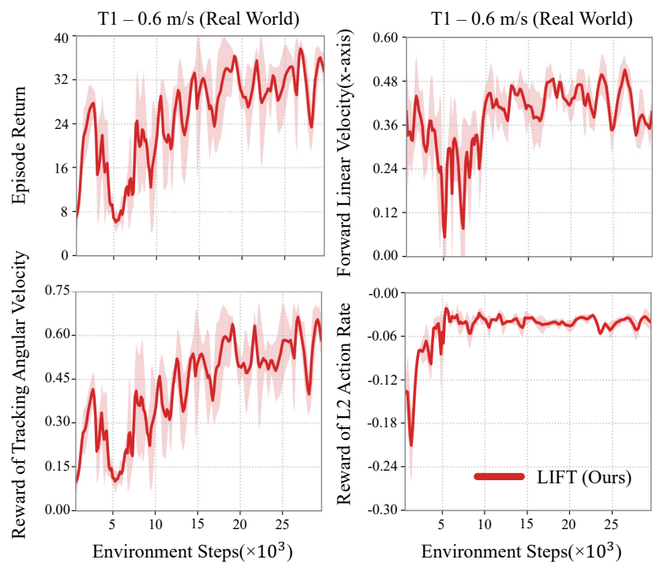


Figure 8: Finetuning performance of LIFT in the Real-World. Across 3 random seeds, LIFT consistently improves episode return, forward velocity tracking, and angular-velocity tracking, while reducing the action-rate penalty. Note that the velocity tracking error (target at 0.6 m/s) might come from the noisy base-velocity estimation from the onboard IMU accelerator.

A.2 预训练实验

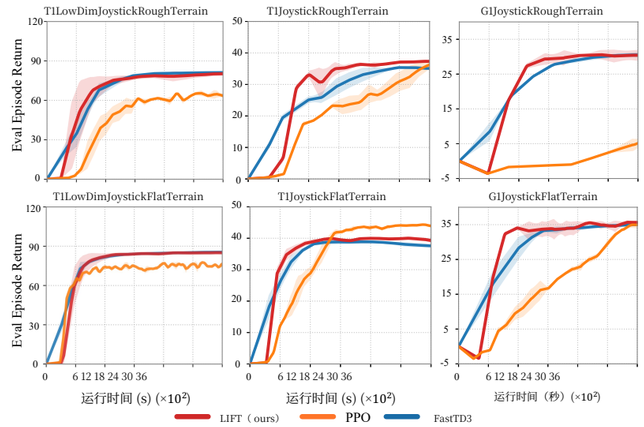


图 7：LIFT（红色）、PPO（橙色）和 FastTD3（蓝色）在六项人形机器人任务上的预训练性能比较（第一行：三种机器人配置在崎岖地形上；第二行：三种机器人配置在平坦地形上）。结果显示了 8 个随机种子的平均值。

A.3 真实世界中的微调实验

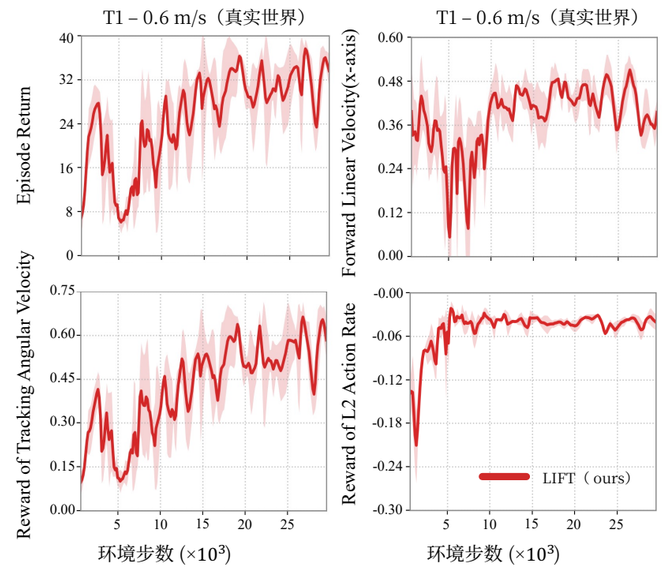


图 8：LIFT 在真实世界中的微调性能。在 3 个随机种子下，LIFT 持续提升了回合回报、前进速度跟踪和角速度跟踪，同时降低了动作速率惩罚。请注意，速度跟踪误差（目标为 0.6 m/s）可能源于机载 IMU 加速度计产生的噪声基座速度估计。

A.4 THE EFFECT OF HYPERPARAMETER

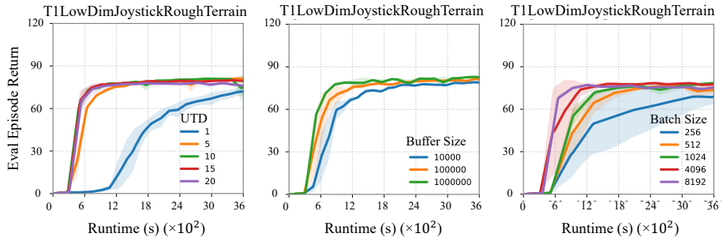


Figure 9: Effect of pretraining hyperparameters on Low dimensional Booster T1. From left to right: UTD ratio, buffer size, and batch size. UTD values compared: 1, 5, 10, 15, 20; buffer sizes: 10,000, 100,000, 1,000,000; batch sizes: 256, 512, 1,024, 4,096, 8,192. Larger UTD and batch sizes accelerate convergence; increasing buffer size improves learning speed but at the cost of much higher GPU memory usage.

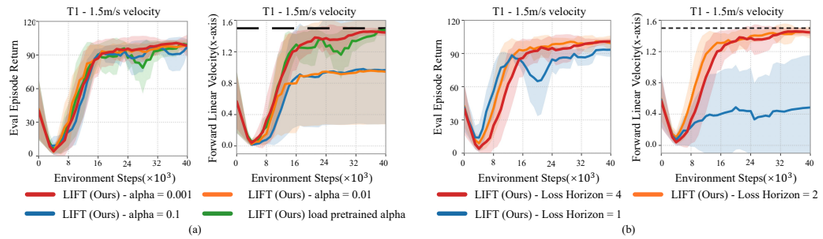


Figure 10: Impact of the entropy coefficient α and autoregressive loss horizon on finetuning performance (Booster T1, target speed $= 1.5$ m/s). Curves show evaluation episode return and body forward velocity. Larger α values degrade performance, leading to less stable convergence and lower final forward velocity. Using the pretraining α gives reasonable results, while smaller α yields more stable learning. Horizon $= 1$ sometimes fails to reach the target velocity, while horizons $= 2$ and $= 4$ ensure stable convergence, indicating that multi-step autoregressive prediction improves world-model training and stabilizes policy finetuning.

A.4 超参数的影响

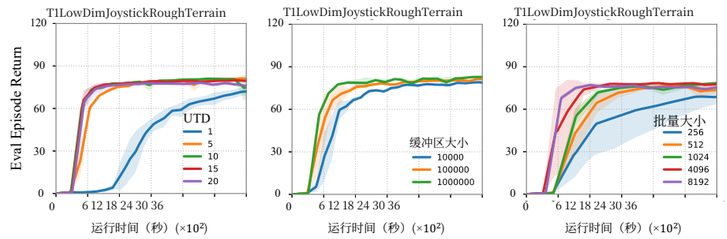


图 9：预训练超参数对低维增强器 T1 的影响。从左到右依次为：UTD 比率、缓冲区大小和批量大小。比较的 UTD 值为：1、5、10、15、20；缓冲区大小为：10,000、100,000、1,000,000；批量大小为：256、512、1,024、4,096、8,192。较大的 UTD 和批量大小可加速收敛；增加缓冲区大小可提高学习速度，但代价是 GPU 内存使用量显著增加。

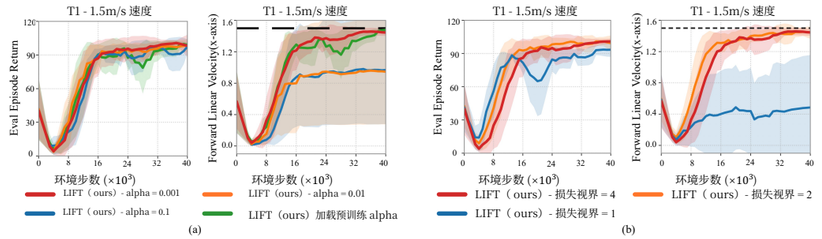
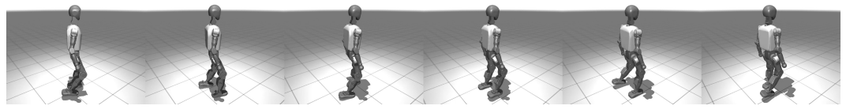


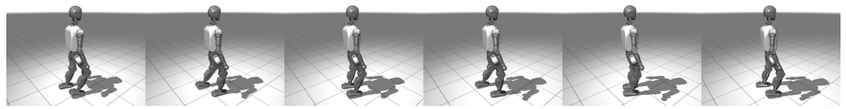
图 10：熵系数 α 和自回归损失范围对微调性能的影响（Booster T1，目标速度 $= 1.5$ m/s）。曲线展示了评估回合回报和身体前进速度。较大的 α 值会降低性能，导致收敛不稳定且最终前进速度较低。使用预训练 α 可获得合理结果，而较小的 α 则带来更稳定的学习。时间范围 $= 1$ 有时无法达到目标速度，而时间范围 $= 2$ 和 $= 4$ 能确保稳定收敛，这表明多步自回归预测改善了世界模型训练并稳定了策略微调。

A.5 PRETRAINING EXPERIMENTS

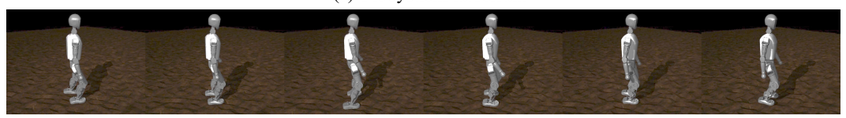


(a) T1LowDimJoystickFlatTerrain

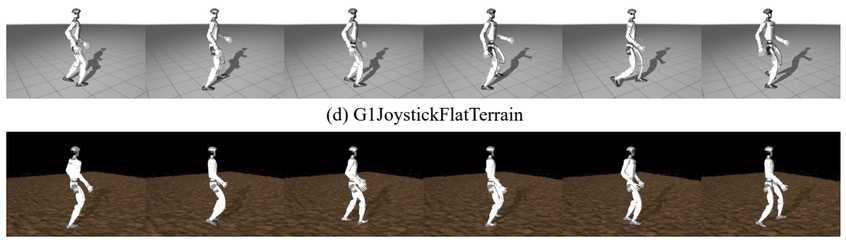
(b) T1LowDimJoystickRoughTerrain



(c) T1JoystickFlatTerrain



(d) T1JoystickRoughTerrain

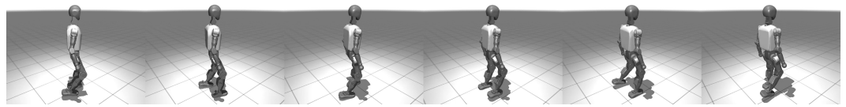


(d) G1JoystickFlatTerrain

(f) G1JoystickRoughTerrain

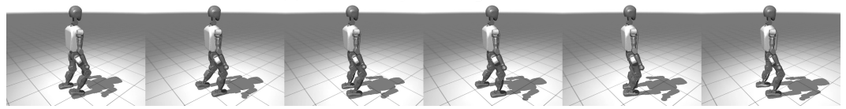
Figure 11: Gaits of LIFT across different tasks in the MuJoCo Playground.

A.5 预训练实验



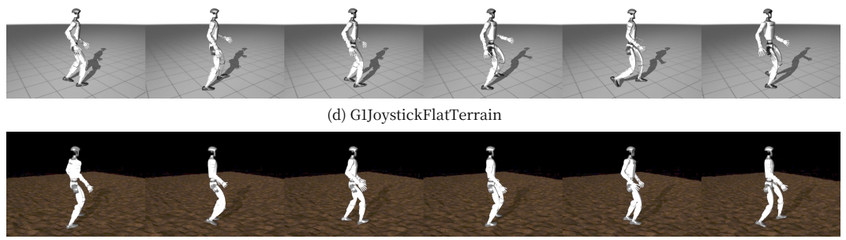
(a) T1LowDimJoystickFlatTerrain

(b) T1LowDimJoystickRoughTerrain



(c) T1JoystickFlatTerrain

(d) T1JoystickRoughTerrain



(d) G1JoystickFlatTerrain

(f) G1JoystickRoughTerrain

图 11：LIFT 在 MuJoCo Playground 不同任务中的步态。

A.6 Real World Experiments

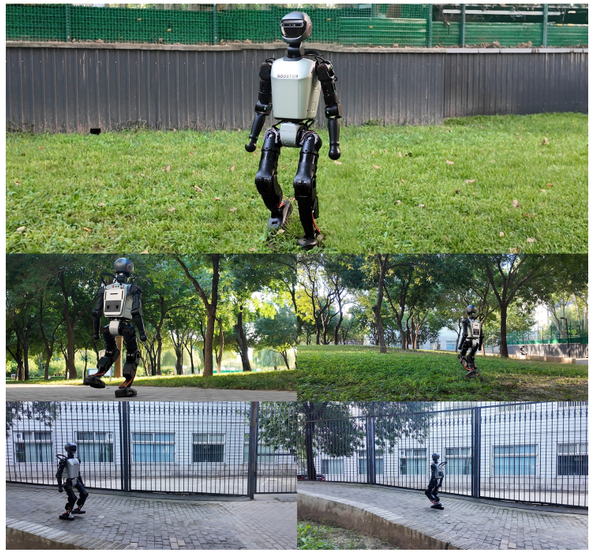


Figure 12: Sim-to-real reinforcement learning with LIFT.

Figure 13: Sim-to-real gait comparison in indoor and outdoor environments.

A.6 真实世界实验

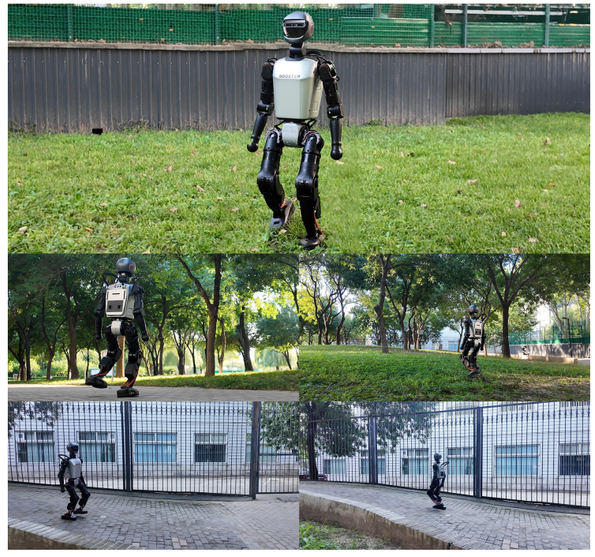


图 12：使用 LIFT 进行仿真到现实强化学习。

图 13：室内和室外环境中的仿真到真实步态对比。

A.7 FINETUNING: G1 EXPERIMENTS

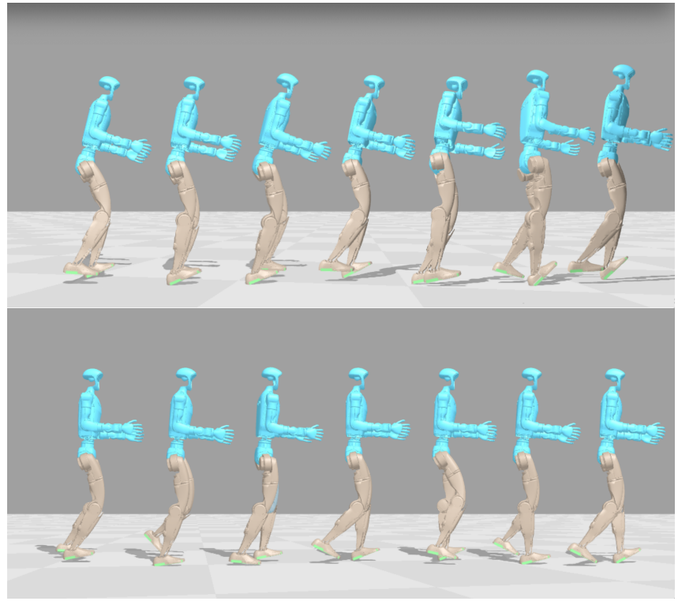


Figure 14: Sim-to-sim transfer and fine-tuning results for the Unitree G1 in Brax with a target velocity of 1.5 m/s. **Top:** Policy before fine-tuning, exhibiting instability and shuffling gait. **Bottom:** Policy after fine-tuning, demonstrating a stable, human-like walking gait with reduced torso pitch.

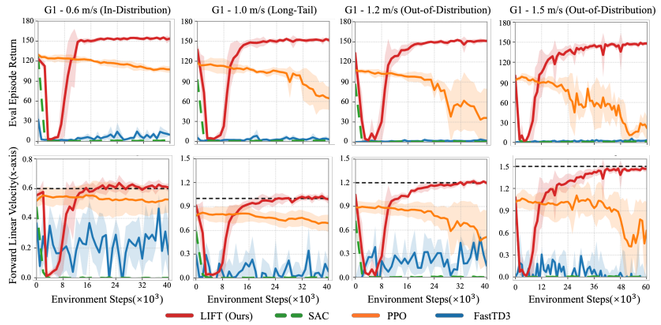


Figure 15: Finetuning results on the Unitree G1 in Brax. LIFT consistently improves policy behavior and reward performance. At a target speed of 0.6 m/s, body oscillations are significantly reduced, while at 1.5 m/s, the policy initially exhibits unstable motion in sim2sim but, after finetuning, successfully stands and walks.

A.7 微调：G1 实验

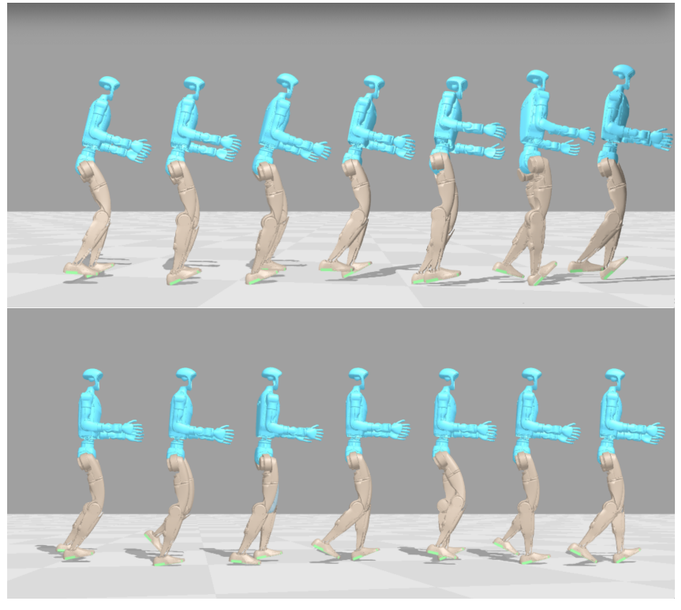


图 14：Unitree G1 在 Brax 中以 1.5 m/s 目标速度进行的仿真到仿真迁移和微调结果。**上图：**微调前的策略，表现出不稳定性和拖步步态。**下图：**微调后的策略，展现出稳定的类人行进步态，且躯干俯仰减小。

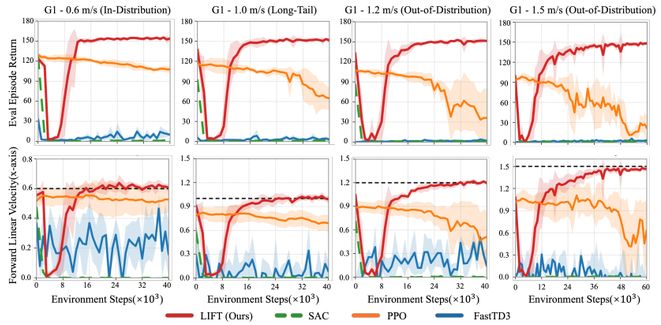


图 15：Unitree G1 在 Brax 中的微调结果。LIFT 持续改进了策略行为和奖励性能。在 0.6 m/s 的目标速度下，身体振荡显著减少；而在 1.5 m/s 时，策略在仿真到仿真中最初表现出不稳定运动，但经过微调后成功站立并行走。

A.8 NON-LOCOMOTION TASKS

Our framework is built around learning a world model of the robot's state dynamics, so any task whose reward depends only on the robot proprioceptive state can, in principle, be handled directly. This includes different locomotion gaits, whole-body motion tracking, and balance under disturbances, since as long as the reward can be computed from the state, the policy can be finetuned entirely through world-model rollouts.

Whole body motion tracking Tasks. To better demonstraste this potential, we extended our pretraining pipeline from velocity-tracking locomotion to BeyondMimic-style whole-body tracking (Liao et al., 2025). We reimplemented the observation and reward structure of BeyondMimic in JAX within MuJoCo Playground and used the Unitree motion dataset (LAFAN1) to pretrain a whole-body tracking policy for the Unitree G1 humanoid. Video demonstrations are available on our project website. Due to the engineering effort required to reproduce consistent observation/reward definitions and contact handling across MuJoCo and Brax, and to integrate these components into our world-model finetuning pipeline, we were able to complete only the pretraining stage within the current timeframe. We therefore present these whole-body tracking results as preliminary evidence of LIFT's broader applicability. This limitation is practical rather than conceptual: once the corresponding Brax environment is finalized or real Unitree G1 hardware is available, the same stage-(iii) finetuning procedure can be directly applied.

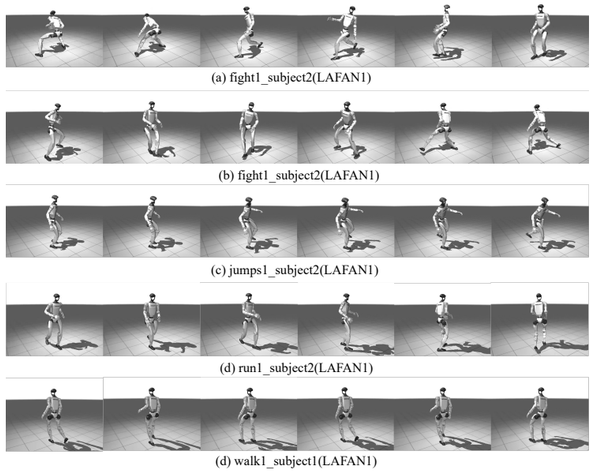


Figure 16: Gaits of LIFT across different whole body motion tracking tasks in the MuJoCo Playground.

Object-centric Tasks. For tasks involving external objects (e.g., kicking a ball), the current framework would need to be extended to also model object dynamics, for example by augmenting the state with object pose and using simple physical priors such as conservation of momentum. This is feasible but requires additional engineering and environment design, so we explicitly leave it as future work. Likewise, obstacle avoidance and more complex navigation are more naturally handled at a higher level: once a reliable low-level tracking controller is available (as in our setup), a high-level policy can be trained to output velocity or pose targets, potentially with its own high-level world model as explored in hierarchical model-based RL (Hansen et al., 2024). Extending LIFT to hierarchical and object-centric tasks is a promising direction, but beyond the scope of this paper.

A.8 非运动任务

我们的框架围绕学习机器人状态动力学的世界模型构建，因此原则上，任何仅依赖于机器人本体感受状态的奖励的任务都可以直接处理。这包括不同的运动步态、全身运动跟踪以及在扰动下的平衡，因为只要奖励可以从状态中计算得出，策略就可以完全通过世界模型 rollout 进行微调。

全身运动跟踪任务。为了更好地展示这一潜力，我们将预训练流程从速度跟踪运动扩展到了 BeyondMimic 风格的全身跟踪（Liao 等，2025）。我们在 MuJoCo Playground 中使用 JAX 重新实现了 BeyondMimic 的观测与奖励结构，并利用 Unitree 运动数据集（LAFAN1）为 Unitree G1 人形机器人预训练了一个全身跟踪策略。视频演示可在我们的项目网站上查看。由于需要在 MuJoCo 和 Brax 之间复现一致的观测/奖励定义和接触处理，并将这些组件集成到我们的世界模型微调流程中，所需的工程工作量较大，因此在当前时间范围内，我们仅完成了预训练阶段。因此，我们将这些全身跟踪结果作为 LIFT 具有更广泛适用性的初步证据。这一限制是实际层面的而非概念层面的：一旦相应的 Brax 环境最终确定或真实 Unitree G1 硬件可用，相同的第 (iii) 阶段微调流程即可直接应用。

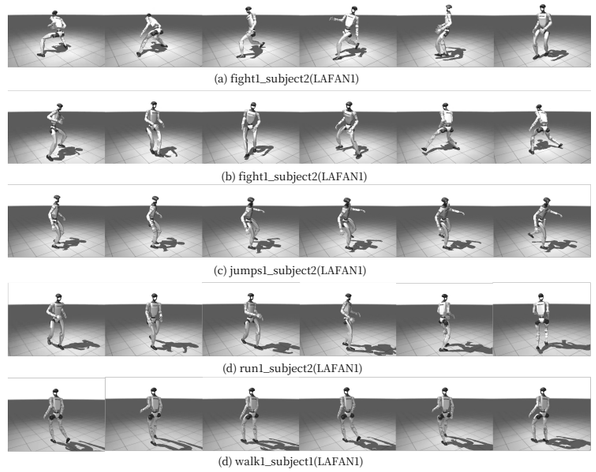


图 16：LIFT 在 MuJoCo Playground 中不同全身运动跟踪任务下的步态。

以物体为中心的任务。对于涉及外部物体的任务（例如踢球），当前的框架需要扩展以同时对物体动力学进行建模，例如通过将物体位姿 augment 到状态中，并使用简单的物理先验（如动量守恒）。这在技术上是可行的，但需要额外的工程工作和环境设计，因此我们明确将其留作未来工作。同样，避障和更复杂的导航更自然地在高层进行处理：一旦有了可靠的底层跟踪控制器（如我们的设置中所用），就可以训练一个高层策略来输出速度或位姿目标，并可能配备其自身的高层世界模型，正如在分层基于模型的强化学习中所探索的那样（Hansen 等，2024）。将 LIFT 扩展到分层和以物体为中心的任务是一个有前景的方向，但超出了本文的范围。

B TRAINING DETAILS

Algorithm 1 The Finetuning phase of LIFT

Require: Pretrained policy π; pretrained world model wm; world-model replay buffer D_{wm}; batch size B; SAC replay buffer D_{sac}; maximum episode length T_{ep}; number of training iterations num_{train}; world-model rollout horizon H_{wm}.

while π not converged **do** $s_0 = env.reset()$
  for $i \leftarrow 1$ to T_{ep} **do**
    $a_t \sim \pi(\cdot|s_t)$
    $s_{t+1}, r_{t+1} = env.step(s_t, a_t)$
    $D_{wm} \leftarrow D_{wm} \bigcup \{s_t, a_t, s_{t+1}, r_{t+1}\}$
    $D_{sac} \leftarrow D_{sac} \bigcup \{s_t, a_t, s_{t+1}, r_{t+1}\}$
  end for
  Sample data from D and finetune the world model using equation 19.
  for $g \leftarrow 1$ to num_{train} **do** // num_{train} *is linearly increased from 10 to 1000*
    Sample a batch of (Batch size B) initial state s_0 from the the D_{wm}.
    for $i \leftarrow 1$ to H_{wm} **do** // H_{wm} *is linearly increased from 1 to 20*
      $a_t \sim \pi(\cdot|s_t)$
      $s_{t+1}, r_{t+1} = wm.step(s_t, a_t)$ // *predict the next state and reward via world model.*
      $D_{sac} \leftarrow D_{sac} \bigcup \{s_t, a_t, s_{t+1}, r_{t+1}\}$
    end for
    Sample the transition from the D_{sac}, and update the SAC actor and critic.
  end for
end while
return π

B.1 PRETRAINING DETAILS OF SAC

Training Objective We pretrain the SAC policy in MuJoCo Playground. The critics minimize the entropy-regularized Bellman residual:

$$\mathbb{E}_{(s_t^p, a_t, r_{t+1}, s_{t+1}^p)\sim\mathcal{D},\, a_{t+1}\sim\pi_\theta(\cdot|s_{t+1})}\big[Q_\psi(s_t^p, a_t) - (r_{t+1} + \gamma(\min_{i=1,2} Q_{\bar{\psi}_i}(s_{t+1}^p, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1}|s_{t+1})))^2\big], \tag{6}$$

where $Q_{\bar{\psi}_i}$ denotes the target critics. Transitions are sampled from the replay buffer $\mathcal{D}$. The actor is updated by maximizing the entropy-augmented Q-value:

$$\mathbb{E}_{s_t, s_t^p\sim\mathcal{D},\, a_t\sim\pi_\theta(\cdot|s_t)}[\alpha \log \pi_\theta(a_t|s_t) - \min_{i=1,2} Q_{\psi_i}(s_t^p, a_t)]. \tag{7}$$

The entropy coefficient α is adjusted to match a target entropy $\bar{\mathcal{H}}$ via

$$\mathbb{E}_{s_t\sim\mathcal{D},\, a_t\sim\pi_\theta(\cdot|s)}[-\alpha(\log \pi_\theta(a_t|s_t) + \bar{\mathcal{H}})]. \tag{8}$$

Finally, the target critics are updated by Polyak averaging: $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$, where $\tau \in (0, 1)$ is the update rate.

Hyperparameter-Tuning We employ the `Optuna` framework (Akiba et al., 2019) for systematic hyperparameter tuning, leveraging the `CmaEsSampler` for parameter exploration in combination with a `PatientPruner` (built on top of a `SuccessiveHalvingPruner`) to automatically terminate trials with poor early performance. The tuned hyperparameters include the number of gradient updates per step (UTD), replay buffer size, entropy coefficient α, discount factor γ, learning rates of the actor, critic, and temperature parameter α, batch size, reward scaling, number of parallel environments, and Polyak coefficient τ. The optimization objective is the episode reward obtained during policy evaluation. On the `T1LowDimJoystick` tasks, we conducted approximately ten hours of hyperparameter search on a single NVIDIA RTX 4090 GPU. The results indicate that tuning reduces the convergence time from about seven hours to only half an hour. On the `G1Joystick` and `T1Joystick` tasks, we carried out a similar ten-hour search using eight NVIDIA RTX 4090 GPUs, which led to improved reward performance and more robust convergence. Table 1 lists the tuned hyperparameters.

B 训练细节

Algorithm 1 The Finetuning phase of LIFT

Require: Pretrained policy π; pretrained world model wm; world-model replay buffer D_{wm}; batch size B; SAC replay buffer D_{sac}; maximum episode length T_{ep}; number of training iterations num_{train}; world-model rollout horizon H_{wm}.

while π not converged **do** $s_0 = env.reset()$
  for $i \leftarrow 1$ to T_{ep} **do**
    $a_t \sim \pi(\cdot|s_t)$
    $s_{t+1}, r_{t+1} = env.step(s_t, a_t)$
    $D_{wm} \leftarrow D_{wm} \bigcup \{s_t, a_t, s_{t+1}, r_{t+1}\}$
    $D_{sac} \leftarrow D_{sac} \bigcup \{s_t, a_t, s_{t+1}, r_{t+1}\}$
  end for
  Sample data from D and finetune the world model using equation 19.
  for $g \leftarrow 1$ to num_{train} **do** // num_{train} *is linearly increased from 10 to 1000*
    Sample a batch of (Batch size B) initial state s_0 from the the D_{wm}.
    for $i \leftarrow 1$ to H_{wm} **do** // H_{wm} *is linearly increased from 1 to 20*
      $a_t \sim \pi(\cdot|s_t)$
      $s_{t+1}, r_{t+1} = wm.step(s_t, a_t)$ // *predict the next state and reward via world model.*
      $D_{sac} \leftarrow D_{sac} \bigcup \{s_t, a_t, s_{t+1}, r_{t+1}\}$
    end for
    Sample the transition from the D_{sac}, and update the SAC actor and critic.
  end for
end while
return π

B.1 SAC 的预训练细节

训练目标我们在 MuJoCo Playground 中预训练 SAC 策略。评论家最小化熵正则化贝尔曼残差：

$$\mathbb{E}_{(s_t^p, a_t, r_{t+1}, s_{t+1}^p)\sim\mathcal{D},\, a_{t+1}\sim\pi_\theta(\cdot|s_{t+1})}\big[Q_\psi(s_t^p, a_t) - (r_{t+1} + \gamma(\min_{i=1,2} Q_{\bar{\psi}_i}(s_{t+1}^p, a_{t+1}) - \alpha \log \pi_\theta(a_{t+1}|s_{t+1})))^2\big], \tag{6}$$

其中 $Q_{\bar{\psi}_i}$ 表示目标评论家。转换样本从回放缓冲区 $\mathcal{D}$中采样。演员通过最大化熵增强 Q 值进行更新：

$$\mathbb{E}_{s_t, s_t^p\sim\mathcal{D},\, a_t\sim\pi_\theta(\cdot|s_t)}[\alpha \log \pi_\theta(a_t|s_t) - \min_{i=1,2} Q_{\psi_i}(s_t^p, a_t)]. \tag{7}$$

¯熵系数 α 被调整以匹配目标熵 $\mathcal{H}$，通过

$$\mathbb{E}_{s_t\sim\mathcal{D},\, a_t\sim\pi_\theta(\cdot|s)}[-\alpha(\log \pi_\theta(a_t|s_t) + \bar{\mathcal{H}})]. \tag{8}$$

¯最后，目标评论家通过 Polyak 平均进行更新：$\psi \leftarrow \tau\psi + (1-\tau)\psi$，其中 $\tau \in (0, 1)$ 是更新率。

超参数调优我们采用 Optuna 框架（Akiba 等人，2019）进行系统的超参数调优，利用 CmaEsSampler 进行参数探索，并结合 PatientPruner（基于 SuccessiveHalvingPruner 构建）自动终止早期表现不佳的试验。所调优的超参数包括每步梯度更新次数（UTD）、回放缓冲区大小、熵系数 α、折扣因子 γ、演员、评论家和温度参数 α 的学习率、批量大小、奖励缩放、并行环境数量以及 Polyak 系数 τ。优化目标是在策略评估期间获得的回合奖励。在 T1LowDimJoystick 任务上，我们在单块 NVIDIA RTX 4090 GPU 上进行了约十小时的超参数搜索。结果表明，调优将收敛时间从约七小时缩短至仅半小时。在 G1Joystick 和 T1Joystick 任务上，我们使用八块 NVIDIA RTX 4090 GPU 进行了类似的十小时搜索，从而实现了更优的奖励性能和更鲁棒的收敛。表 1 列出了调优后的超参数。

Table 1: LIFT Hyperparameter Configurations for MuJoCo Playground Environments

Hyperparameter	**T1LowDimJoystick***	**G1/T1 Joystick***	**T1JoystickFlatTerrain**
Parallel environments	1,000	4,096	4,096
Batch size	1,024	16,384	16,384
Gradient updates/step	9	19	16
Discount factor (γ)	0.987	0.982	0.982
Reward scaling	1.0	16.0	32.0
Soft update (τ)	0.024	0.002	0.007
Actor learning rate	1.03e-4	1.70e-4	2.05e-4
Critic learning rate	1.00e-4	1.88e-4	1.37e-4
Alpha learning rate	9.97e-3	6.44e-4	6.41e-4
Target entropy coefficient	0.50	0.40	0.17
Initial log α	-3.35	-3.00	-6.05
Policy hidden layers	(512, 256, 128)	(512, 256, 128)	(512, 256, 128)
Q-network hidden layers	(1024, 512, 256)	(1024, 512, 256)	(1024, 512, 256)
Activation	swish	swish	swish
Max replay size ($\times 10^6$)	1.0	1.0	1.0
Min replay size	8,192	8,192	8,192
Observation normalization	True	True	True
Action repeat	1	1	1

B.2 Training Details of World Model

(a) From Privileged State to Brax Generalized State. We build the *generalized coordinates/velocities* of Brax using the privileged state and denote them simply by $(q_t, \dot{q}_t)$ for the rest of the section:

$$q_t = \big[p_t,\ n_t,\ q_{j,t}\big], \quad p_t = (0, 0, h_t); \tag{9}$$

$$\dot{q}_t = \big[v_t^{\mathrm{w}},\ \omega_t^{\mathrm{b}},\ \dot{q}_{j,t}\big], \quad v_t^{\mathrm{w}} = R(n_t)\, v_t^{\mathrm{b}}, \tag{10}$$

where $R(n) \in \mathrm{SO}(3)$ be the rotation matrix from the base quaternion n (body→world), with $R(q)^\top$ its inverse. Brax exposes differentiable rigid-body dynamics primitives—mass matrix $M(q)$, Coriolis/centrifugal terms $C(q, \dot{q})$, and gravity $G(q)$—thereby avoiding any reimplementation of dynamics and keeping the entire rollout–loss pipeline end-to-end differentiable.

(b) From Privileged State and Action to Torque (repeat N substeps). The policy action is converted to motor torques τ_t^{m} via a PD controller: $\tau_t^{\mathrm{m}} = K_p\,(a_t + q_t^{\mathrm{default}} - q_{j,t}) + K_d\,(0 - \dot{q}_{j,t})$, where K_p and K_d are stiffness and damping; q_t^{default} is default/standing joint position. The external torque τ_t^e come from the network $\tau_\phi(s_t^p, a_t)$. We assume the control rate f_{ctrl} is lower than the simulator/collection rate f_{sim}. Hence each control interval contains $N = \frac{f_{\mathrm{sim}}}{f_{\mathrm{ctrl}}}$ simulator substeps of size Δt_{sub}, so that the control interval length is $\Delta t = N\,\Delta t_{\mathrm{sub}}$.

(c) One Semi-implicit Euler Step (repeat N substeps). For substep $k = 0, \ldots, N-1$ with step size Δt_{sub}:

$$\ddot{q}_t = M^{-1}(q_t)[\tau_t^m + \tau_t^e - C(q_t, \dot{q}_t) - G(q_t)], \tag{11}$$

$$\dot{q}_{t+\Delta t_{\mathrm{sub}}} = \dot{q}_t + \Delta t_{\mathrm{sub}}\, \ddot{q}_t, \tag{12}$$

$$q_{j,t+\Delta t_{\mathrm{sub}}} = q_{j,t} + \dot{q}_{t+\Delta t_{\mathrm{sub}}} \tag{13}$$

$$p_{t+\Delta t_{\mathrm{sub}}} = p_t + \Delta t_{\mathrm{sub}}\, v^{\mathrm{w}}_{t+\Delta t_{\mathrm{sub}}}, \tag{14}$$

$$n_{t+\Delta t_{\mathrm{sub}}} = \mathrm{normalize}\Big(n_t \otimes \mathrm{quat}\big(\widehat{\omega^{\mathrm{b}}_{t+\Delta t_{\mathrm{sub}}}},\ \|\omega^{\mathrm{b}}_{t+\Delta t_{\mathrm{sub}}}\|\,\Delta t_{\mathrm{sub}}\big)\Big). \tag{15}$$

Here, $\widehat{\omega} = \omega/\|\omega\|$ is the unit rotation axis; $\mathrm{quat}(\widehat{\omega}, \theta) = [\cos(\frac{\theta}{2}), \widehat{\omega}\sin(\frac{\theta}{2})]$ is the axis–angle increment as a unit quaternion; $\otimes$ denotes quaternion multiplication; $\mathrm{normalize}(\cdot)$ re-unitizes the quaternion to avoid numerical drift. After N substeps, we obtain:

$$q_{t+\Delta t} = \big[p_{t+\Delta t},\ n_{t+\Delta t},\ q_{j,t+\Delta t}\big], \tag{16}$$

$$\dot{q}_{t+\Delta t} = \big[v^{\mathrm{w}}_{t+\Delta t},\ \omega^{\mathrm{b}}_{t+\Delta t},\ \dot{q}_{j,t+\Delta t}\big]. \tag{17}$$

表 1：LIFT 在 MuJoCo Playground 环境中的超参数配置

超参数	**T1LowDimJoystick***	**G1/T1 Joystick***	**T1JoystickFlatTerrain**
并行环境	1,000	4,096	4,096
批量大小	1,024	16,384	16,384
每步梯度更新	9	19	16
折扣因子 (γ)	0.987	0.982	0.982
奖励缩放	1.0	16.0	32.0
软更新 (τ)	0.024	0.002	0.007
Actor 学习率	1.03e-4	1.70e-4	2.05e-4
Critic 学习率	1.00e-4	1.88e-4	1.37e-4
Alpha 学习率	9.97e-3	6.44e-4	6.41e-4
目标熵系数	0.50	0.40	0.17
初始 log α	-3.35	-3.00	-6.05
策略隐藏层	(512, 256, 128)	(512, 256, 128)	(512, 256, 128)
Q 网络隐藏层	(1024, 512, 256)	(1024, 512, 256)	(1024, 512, 256)
激活函数	Swish 激活函数	Swish 激活函数	Swish 激活函数
最大回放缓冲区大小 ($\times 10^6$)	1.0	1.0	1.0
最小回放缓冲区大小	8,192	8,192	8,192
观测归一化	True	True	True
动作重复	1	1	1

B.2 WORLDMODEL 训练细节

(a) 从特权状态到 *Brax* 广义状态。我们利用特权状态构建 Brax 的广义坐标/速度，并在本节余下部分将其简记为 $(q_t, \dot{q}_t)$：

$$q_t = \big[p_t,\ n_t,\ q_{j,t}\big], \quad p_t = (0, 0, h_t); \tag{9}$$

$$\dot{q}_t = \big[v_t^{\mathrm{w}},\ \omega_t^{\mathrm{b}},\ \dot{q}_{j,t}\big], \quad v_t^{\mathrm{w}} = R(n_t)\, v_t^{\mathrm{b}}, \tag{10}$$

其中 $R(n) \in \mathrm{SO}(3)$ 为从基四元数 n（本体到世界）导出的旋转矩阵，$R(q)^\top$ 为其逆矩阵。Brax 提供了可微刚体动力学原语——质量矩阵 $M(q)$、科里奥利力/离心力项 $C(q, \dot{q})$ 以及重力 $G(q)$——从而避免了对动力学的任何重新实现，并保持整个 rollout-损失流水线端到端可微。

(b) 从特权状态和动作到扭矩（重复 N 子步）。策略动作通过 PD 控制器转换为电机扭矩 τ_{mt}：$\tau_t^{\mathrm{m}} = K_p\,(a_t + q_t^{\mathrm{default}} - q_{j,t}) + K_d\,(0 - \dot{q}_{j,t})$，其中 K_p 和 K_d 分别为刚度和阻尼；q_t^{default} 为默认 / 站立关节位置。外部扭矩 τ_t^e 来自网络 $\tau_\phi(s_t^p, a_t)$。我们假设控制频率 f_{ctrl} 低于仿真器/采集频率 f_{sim}。因此，每个控制间隔包含 $N = \frac{f_{\mathrm{sim}}}{f_{\mathrm{ctrl}}}$ 个大小为 Δt_{sub} 的仿真子步，使得控制间隔长度为 $\Delta t = N\,\Delta t_{\mathrm{sub}}$。

(c) 一个半隐式欧拉步（重复 N 个子步）。对于子步 $k = 0 \ldots N-1$，步长为Δt_{sub}：

$$\ddot{q}_t = M^{-1}(q_t)[\tau_t^m + \tau_t^e - C(q_t, \dot{q}_t) - G(q_t)], \tag{11}$$

$$\dot{q}_{t+\Delta t_{\mathrm{sub}}} = \dot{q}_t + \Delta t_{\mathrm{sub}}\, \ddot{q}_t, \tag{12}$$

$$q_{j,t+\Delta t_{\mathrm{sub}}} = q_{j,t} + \dot{q}_{t+\Delta t_{\mathrm{sub}}} \tag{13}$$

$$p_{t+\Delta t_{\mathrm{sub}}} = p_t + \Delta t_{\mathrm{sub}}\, v^{\mathrm{w}}_{t+\Delta t_{\mathrm{sub}}}, \tag{14}$$

$$n_{t+\Delta t_{\mathrm{sub}}} = \mathrm{normalize}\Big(n_t \otimes \mathrm{quat}\big(\widehat{\omega^{\mathrm{b}}_{t+\Delta t_{\mathrm{sub}}}},\ \|\omega^{\mathrm{b}}_{t+\Delta t_{\mathrm{sub}}}\|\,\Delta t_{\mathrm{sub}}\big)\Big). \tag{15}$$

此处，$\widehat{\omega} = \omega/\|\omega\|$ 为单位旋转轴；$\mathrm{quat}(\widehat{\omega}, \theta) = [\cos(\frac{\theta}{2}), \widehat{\omega}\sin(\frac{\theta}{2})]$为以单位四元数表示的轴角增量；$\otimes$ 表示四元数乘法；$\mathrm{normalize}(\cdot)$ 将四元数重新单位化以避免数值漂移。经过 N 个子步骤后，我们得到：

$$q_{t+\Delta t} = \big[p_{t+\Delta t},\ n_{t+\Delta t},\ q_{j,t+\Delta t}\big], \tag{16}$$

$$\dot{q}_{t+\Delta t} = \big[v^{\mathrm{w}}_{t+\Delta t},\ \omega^{\mathrm{b}}_{t+\Delta t},\ \dot{q}_{j,t+\Delta t}\big]. \tag{17}$$

(d) From $(q_{t+\Delta t}, \dot{q}_{t+\Delta t})$ to the Next Privileged State.:

$$\widehat{s}^{p}_{t+\Delta t}: \left\{ \begin{array}{r} \text{Joints Position/Velocity: } (q_{j,t+\Delta t}, \dot{q}_{j,t+\Delta t}). \\ \text{Base Linear Velocity: } v^{\mathrm{b}}_{t+\Delta t} = R(n_{t+\Delta t})^{\top} v^{\mathrm{w}}_{t+\Delta t}. \\ \text{Base Angular Velocity: } \omega^{\mathrm{b}}_{t+\Delta t}. \\ \text{Quaternion: } n_{t+\Delta t}. \\ \text{Body Height: } h_{t+\Delta t} = p_{t+\Delta t,z}. \end{array} \right\} \tag{18}$$

B.3 Training Details of World Model in Finetuning Stage

Concretely, we sample length-$H{+}1$ sequences $\mathcal{D}_H = \left\{ (s^p_t,\, a_t,\, s^p_{t+1},\, a_{t+1},\, \ldots,\, s^p_{t+H}) \right\}_{b=1}^{B}$, where B is the mini-batch size and H is the loss horizon (e.g., $H{=}4$). Let $\widehat{s}^{p}_{b,t+1}$ be the model's one-step prediction (obtained by differentiable physics given $(s^p_{b,t}, a_{b,t})$), $s^p_{b,t+1}$ the target, and $\log \sigma^2_{b,t}$ the model's elementwise log-variance. We minimize an auto-regressive diagonal-Gaussian negative log-likelihood over an H-step unroll:

$$\mathcal{L}_\phi = \frac{1}{B\,H} \sum_{b=1}^{B} \sum_{t=0}^{H-1} \Big[\big(\widehat{s}^{p}_{b,t+1} - s^p_{b,t+1}\big)^2 \odot \exp\big(-\log \sigma^2_{b,t}\big) + \log \sigma^2_{b,t} \Big], \tag{19}$$

where $\odot$ denotes elementwise multiplication, $\widehat{s}^{p}_{b,t+1}$ is the model's next-step prediction given $(s^p_{b,t}, a_{b,t})$, $s^p_{b,t+1}$ is the target, and $\log \sigma^2_{b,t}$ is the elementwise log-variance. Gradients backpropagate through the entire H-step unroll.

B.4 Training Details of Policy in Finetuning Stage

A key advantage of our physics-informed design is that $\widehat{s}^{p}_{t}$ exposes kinematic and dynamic signals that enable *exact*, simulator-consistent reward computation without training a reward model. We thus define a per-step reward:

$$r_{t+1} = \sum_k w_k\, r_k\big(\widehat{s}^{p}_{t}, a_t, \widehat{s}^{p}_{t+1}\big), \tag{20}$$

with each r_k computed analytically. For example, foot orientation stability follows directly from each foot's roll angle and is penalized by

$$r_{\text{feet-roll}} = - \sum_{f \in \{\text{Left foot,Right foot}\}} \big(\varphi^f\big)^2, \tag{21}$$

where φ^f is the roll of foot f extracted from the link quaternion in $\widehat{s}^{p}_{t}$. Table 1 lists the LIFT hyperparameters in finetuning.

(d) 从 $(q_{t+\Delta t}, \dot{q}_{t+\Delta t})$到下一个特权状态:

$$\widehat{s}^{p}_{t+\Delta t}: \left\{ \begin{array}{r} \text{Joints Position/Velocity: } (q_{j,t+\Delta t}, \dot{q}_{j,t+\Delta t}). \\ \text{Base Linear Velocity: } v^{\mathrm{b}}_{t+\Delta t} = R(n_{t+\Delta t})^{\top} v^{\mathrm{w}}_{t+\Delta t}. \\ \text{Base Angular Velocity: } \omega^{\mathrm{b}}_{t+\Delta t}. \\ \text{Quaternion: } n_{t+\Delta t}. \\ \text{Body Height: } h_{t+\Delta t} = p_{t+\Delta t,z}. \end{array} \right\} \tag{18}$$

B.3 微调阶段世界模型的训练细节

具体而言，我们采样长度为 $H{+}1$ 的序列 $\mathcal{D}_H = \left\{ (s^p_t,\, a_t,\, s^p_{t+1},\, a_{t+1},\, \ldots,\, s^p_{t+H}) \right\}_{b=1}^{B}$，其中 B 是小批量大小，H 是损失视界（例如 $H{=}4$）。设 $\widehat{s}^{p}_{b,\,t+1}$ 为模型的一步预测（在给定 $(s^p_{b,t}, a_{b,t})$ $s^p_{b\,t+1}$ 目标的情况下通过可微物理获得），而 $\log\, \sigma^2_{b,t}$ 为模型的逐元素对数方差。我们在一个 H 步展开上最小化自回归对角高斯负对数似然:

$$\mathcal{L}_\phi = \frac{1}{B\,H} \sum_{b=1}^{B} \sum_{t=0}^{H-1} \Big[\big(\widehat{s}^{p}_{b,t+1} - s^p_{b,t+1}\big)^2 \odot \exp\big(-\log \sigma^2_{b,t}\big) + \log \sigma^2_{b,t} \Big], \tag{19}$$

其中 $\odot$ 表示逐元素乘法，$\widehat{s}^{p}_{b\,t+1}$ 是在给定 $(s^p_{b,t}, a_{b,t})$ $s^p_{b\,t+1}$ 目标的情况下模型的下一步预测，而 $\log \sigma^2_{b,\,t}$ 是逐元素对数方差。梯度在整个 H 步展开中进行反向传播。

B.4 策略微调阶段的训练细节

我们物理信息设计的一个关键优势在于，$\widehat{s}^{p}_{t}$ 提供了运动学和动力学信号，使得无需训练奖励模型即可进行精确且与模拟器一致的奖励计算。因此，我们定义了一个逐步奖励:

$$r_{t+1} = \sum_k w_k\, r_k\big(\widehat{s}^{p}_{t}, a_t, \widehat{s}^{p}_{t+1}\big), \tag{20}$$

其中每个 r_k 均通过解析方式计算。例如，足部方向稳定性直接由每只脚的横滚角得出，并受到惩罚

$$r_{\text{feet-roll}} = - \sum_{f \in \{\text{Left foot,Right foot}\}} \big(\varphi^f\big)^2, \tag{21}$$

其中 φ^f 是从 $\widehat{s}^{p}_{t}$ 中的连杆四元数提取的脚 f 的翻滚角。表 1 列出了微调中的 LIFT 超参数。

Table 2: Finetune Hyperparameter of LIFT

Hyperparameter	**Actor-Critic**	**World Model**
Policy hidden layers	[512, 256, 128]	—
Q-network hidden layers	[1024, 512, 256]	—
World Model hidden size	—	[400,400,400,400]
Activation	swish	swish
Learning rate	2e-4	1e-3
Discount factor (γ)	0.99	—
Batch size	256	200
Soft update (τ)	0.001	—
Initial log α	-7.13	—
Gradient updates/step	20	—
Real data ratio	0.06	—
Init exploration steps	1,000	—
Replay buffer size	400k	60k
Reward scaling	1.0	—
Model trains/epoch	1	—
Env steps/training	1,000	—
Horizon of exploration in world model	1→20 (epochs 0→10)	—
Gradient step per update	10→1000 (epochs 0→4)	—
Convergence criteria	—	0.01 (6 epochs)
Loss horizon	—	4

C Environment Setup

C.1 Preliminary Study - Booster Environment

Task summary. The agent controls a 12-DoF T1 humanoid robot with simplified lower-body kinematics. At each step, the agent outputs a 12D continuous action that perturbs motor target positions relative to a default pose.

State (39D). The state consists of torso quaternion (4), joint angles (12), base linear velocity in body frame (3), base angular velocity in body frame (3), joint velocities (12), gait phase represented by cosine/sine (2), gait progression (1), gait frequency (1), and base height (1).

Privileged state (39D). The privileged state uses the same 39-dimensional observation space as the state.

Action Space. The action space is a continuous 12-dimensional vector, where policy outputs are constrained to the range $[-1, 1]$. Motor targets are computed using PD control:

$$u = K_p(a \cdot a_s + q_{\text{default}} - q) + K_d(\dot{q}_{\text{des}} - \dot{q}) \tag{22}$$

where $a_s = 0.25$ is the action scale factor. The proportional and derivative gains are defined as:

$$K_p = [200, 200, 200, 200, 50, 50, 200, 200, 200, 200, 50, 50]\ K_d = [5, 5, 5, 5, 1, 1, 5, 5, 5, 5, 1, 1]$$

The gains are symmetric for both legs, with higher gains for hip joints (200 for K_p, 5 for K_d) and lower gains for knee joints (50 for K_p, 1 for K_d).

Reward Function Design. Uses a multi-objective weighted reward function:

$$r_t = \Delta t \sum_i s_i r_i(s_t, a_t)$$

Control Architecture. Uses a two-level control strategy:

1. **High-level Policy:** Outputs joint position increments, control frequency 100 Hz

表 2：LIFT 的微调超参数

超参数	**Actor-Critic**	**世界模型**
策略隐藏层	[512, 256, 128]	—
Q 网络隐藏层	[1024, 512, 256]	—
世界模型隐藏层大小	—	[400,400,400,400]
激活函数	Swish 激活函数	Swish 激活函数
学习率	2e-4	1e-3
折扣因子 (γ)	0.99	—
批量大小	256	200
软更新 (τ)	0.001	—
初始日志 α	-7.13	—
每步梯度更新	20	—
真实数据比例	0.06	—
初始化探索步数	1,000	—
回放缓冲区大小	400k	60k
奖励缩放	1.0	—
每轮次模型训练次数	1	—
每次训练环境步数	1,000	—
世界模型中的探索范围	1→20（轮次 0→10）	—
每次更新的梯度步长	10→1000 （轮次 0→4）	—
收敛准则	—	0.01（6 个轮次）
损失视界	—	4

C 环境设置

C.1 初步研究 - Booster 环境

任务摘要。智能体控制一个具有简化下肢运动学的 12 自由度 T1 人形机器人。在每一步中，智能体输出一个 12 维连续动作，该动作相对于默认姿态扰动电机目标位置。

状态（39 维）。状态包括躯干四元数（4）、关节角度（12）、体坐标系中的基座线速度（3）、体坐标系中的基座角速度（3）、关节速度（12）、用余弦/正弦表示的步态相位（2）、步态进程（1）、步态频率（1）以及基座高度（1）。

特权状态（39 维）。特权状态使用与状态相同的 39 维观测空间。

动作空间。动作空间是一个连续的 12 维向量，其中策略输出被约束在范围 $[-1, 1]$ 内。电机目标使用 PD 控制计算：

$$u = K_p(a \cdot a_s + q_{\text{default}} - q) + K_d(\dot{q}_{\text{des}} - \dot{q}) \tag{22}$$

其中 $a_s = 0.25$ 是动作缩放因子。比例和微分增益定义为：

$$K_p = [200, 200, 200, 200, 50, 50, 200, 200, 200, 200, 50, 50]\ K_d = [5, 5, 5, 5, 1, 1, 5, 5, 5, 5, 1, 1]$$

双腿的增益对称，髋关节的增益较高（K_p 为 200，K_d 为 5），膝关节的增益较低（K_p 为 50，K_d 为 1）。

奖励函数设计。采用多目标加权奖励函数：

$$r_t = \Delta t \sum_i s_i r_i(s_t, a_t)$$

控制架构。采用双层控制策略：

1. **高层策略：**输出关节位置增量，控制频率为 100 Hz

Table 3: Reward Function Components and Weights

Reward Term	Description	Default Weight
base_height	Base height reward	0.2
tracking_lin_vel	Linear velocity tracking reward	1.2
ref	Reference trajectory tracking reward	3.0

Gait Generation. Phase-based gait controller:

$$\phi = \text{mod}(t \cdot f_g, 1.0)$$
$$\text{Left Leg Swing Phase} = |\phi - 0.25| < 0.5 \cdot T_{swing}$$
$$\text{Right Leg Swing Phase} = |\phi - 0.75| < 0.5 \cdot T_{swing}$$

where $T_{swing} = 0.2$s is the swing phase duration.

Termination Conditions. Episode terminates under the following conditions:

- Base height $h \notin [0.3, 0.8]$ m
- Base linear velocity > 10.0 m/s or angular velocity > 10.0 rad/s
- Torso roll angle $|\phi| > \pi/4$ or pitch angle $|\theta| > \pi/4$
- Joint position or velocity exceeds mechanical limits

Experimental Setup. In the Preliminary Study (Section A.1):

- Target forward velocity: 0.2 m/s
- Main reward terms: swing leg reference, base height tracking, linear velocity tracking

表 3：奖励函数组件与权重

奖励项	描述	默认权重
基座高度	基座高度奖励	0.2
线速度跟踪	线速度跟踪奖励	1.2
ref	参考轨迹跟踪奖励	3.0

步态生成。基于相位的步态控制器：

$$\phi = \text{mod}(t \cdot f_g,\ 1.0)$$
$$\text{左腿摆动相} = |\phi - 0.25| < 0.5 \cdot T_{swin}\ _g$$
$$\text{右腿摆动相} = |\phi - 0.75| < 0.5 \cdot T_{swin}\ _g$$

其中 $T_{swing} = 0.2$s 为摆动相持续时间。

终止条件。回合在以下条件下终止：

- 基座高度 $h \notin [0.3, 0.8]$ m
- 基座线速度 > 10.0 m/s 或角速度 > 10.0 rad/s
- 躯干横滚角 $|\phi| > \pi/4$ 或俯仰角 $|\theta| > \pi/4$
- 关节位置或速度超出机械限位

实验设置。在初步研究（附录 A.1）中：

- 目标前进速度：0.2 m/s
- 主要奖励项：摆动腿参考、基座高度跟踪、线速度跟踪

C.2 PRETRAINING ENVIRONMENTS

C.2.1 T1LOWDIMJOYSTICK

(`T1LowDimJoystick` as an example to illustrate the pretraining environment setup, `T1Joystick` and `G1Joystick` are omitted for brevity.)

Task summary. The agent controls a 12-DoF T1 humanoid robot with simplified lower-body kinematics. At each step, the agent outputs a 12D continuous action that perturbs motor target positions relative to a default pose. The environment returns noisy states and a scalar reward. Episodes terminate upon falls or numerical errors (NaNs).

State (47D). The state consists of gravity vector in the body frame (3), gyroscope readings (3), joystick command (v_x, v_y, ω) (3), gait phase represented by cosine/sine (2), joint angles relative to the default pose (12), joint velocities scaled by 0.1 (12), and previous action (12).

Privileged state (110D). The privileged state includes all elements of the 47D state, together with additional raw sensor signals (gyroscope, accelerometer, gravity vector, linear velocity, global angular velocity), joint differences, root height, actuator forces, contact booleans, foot velocities, and foot air time.

Action space. The action is a continuous 12-D vector; policy outputs lie in $[-1, 1]$. Motor targets are computed using a PD control law:

$$u = K_p(a + q_{\text{default}} - q) + K_d(0 - \dot{q}) \tag{23}$$

where u is the output motor torque, K_p and K_d are the proportional and derivative gains, respectively, a is the action output from the policy q_{default} is a nominal joint position, q is the current joint position, and $\dot{q}$ is the current joint velocity.

Reward Instantaneous reward: $r_t = \Delta t \sum_i s_i \, r_i(s_t^p, a_t)$, where s_i are term weights and $\Delta t = 0.02$s.

Table 4: Default reward terms (Note: $U[a, b]$ denotes uniform distribution over $[a, b]$)

Term	Description	Scale
survival	Survival bonus per step.	0.25
tracking_lin_vel_x	Track commanded v_x velocity.	1.0
tracking_lin_vel_y	Track commanded v_y velocity.	1.0
tracking_ang_vel	Track commanded yaw rate.	2.0
feet_swing	Reward proper foot swing phase.	3.0
base_height	Penalize deviation from target height (0.68m).	-20.0
orientation	Penalize torso tilt from vertical.	-10.0
torques	Penalize large torques.	-1.0e-4
torque_tiredness	Penalize torque near limits.	-5.0e-3
power	Penalize positive mechanical power.	-1.0e-3
lin_vel_z	Penalize vertical base velocity.	-2.0
ang_vel_xy	Penalize torso roll/pitch rates.	-0.2
dof_vel	Penalize joint velocity.	-1.0e-4
dof_acc	Penalize joint acceleration.	-1.0e-7
root_acc	Penalize root link acceleration.	-1.0e-4
action_rate	Penalize rapid action changes.	-0.5
dof_pos_limits	Penalize joint limit violations.	-1.0
collision	Penalize self-collisions.	-10.0
feet_slip	Penalize slipping contacts.	-0.1
feet_roll	Penalize foot roll angles.	-1.0
feet_yaw_diff	Penalize difference in foot yaw angles.	-1.0
feet_yaw_mean	Penalize deviation from base yaw.	-1.0
feet_distance	Penalize feet being too close.	-10.0

Implementation details

C.2 预训练环境

C.2.1 T1LowDimJoystick

（以 T1LowDimJoystick 为例说明预训练环境设置，T1Joystick 和 G1Joystick 因简洁起见已省略。）

任务摘要。智能体控制一个具有简化下半身运动学的 12 自由度 T1 人形机器人。在每一步，智能体输出一个 12 维连续动作，该动作相对于默认姿态扰动电机目标位置。环境返回噪声状态和标量奖励。当发生跌倒或数值错误（非数值）时，回合终止。

状态（47 维）。状态包括体坐标系中的重力向量（3）、陀螺仪读数（3）、操纵杆指令（v_x，v_y，ω）（3）、由余弦/正弦表示的步态相位（2）、相对于默认姿态的关节角度（12）、缩放 0.1 倍后的关节速度（12），以及上一动作（12）。

特权状态（110 维）。特权状态包含 47 维状态的所有元素，以及额外的原始传感器信号（陀螺仪、加速度计、重力向量、线速度、全局角速度）、关节差异、根高度、执行器力、接触布尔值、足部速度和足部腾空时间。

动作空间。动作是一个连续的 12 维向量；策略输出位于 $[-1, 1]$中。电机目标使用 PD 控制律计算：

$$u = K_p(a + q_{\text{default}} - q) + K_d(0 - \dot{q}) \tag{23}$$

其中 u 是输出电机扭矩，K_p 和 K_d 分别是比例和微分增益，a是来自策略的动作输出，q_{default} 是标称关节位置，q 是当前关节位置，而 $\dot{q}$ 是当前速度。

奖励瞬时奖励：$r_t = \Delta t \sum_i s_i \, r_i(s_t^p, a_t)$，其中 s_i 为项权重，且 Δt =0。02s。

表 4：默认奖励项（注：$U[a, b]$ 表示在 $[a, b]$上的均匀分布）

Term	描述	缩放
生存	每步生存奖励。	0.25
跟踪线速度 x	跟踪指令 v_x 速度。	1.0
跟踪线速度 y	跟踪指令 v_y 速度。	1.0
跟踪角速度	跟踪指令偏航率。	2.0
足部摆动	奖励正确的足部摆动相位。	3.0
基座高度	惩罚与目标高度（0.68 米）的偏差。	-20.0
朝向	对躯干偏离垂直方向的倾斜进行惩罚。	-10.0
扭矩	对过大的扭矩进行惩罚。	-1.0e-4
扭矩疲劳	对接近极限的扭矩进行惩罚。	-5.0e-3
功率	惩罚正向机械功率。	-1.0e-3
z 轴线速度	惩罚垂直基座速度。	-2.0
xy 轴角速度	惩罚躯干横滚/俯仰速率。	-0.2
自由度速度	惩罚关节速度。	-1.0e-4
自由度加速度	对关节加速度进行惩罚。	-1.0e-7
根节点加速度	对根连杆加速度进行惩罚。	-1.0e-4
动作速率	惩罚快速动作变化。	-0.5
自由度位置限制	惩罚关节限位违反。	-1.0
碰撞	惩罚自碰撞。	-10.0
足部打滑	惩罚滑动接触。	-0.1
足部横滚角	惩罚脚部滚动角度。	-1.0
足部偏航角差	惩罚足部偏航角的差异。	-1.0
足部偏航角均值	惩罚偏离基座偏航角的行为。	-1.0
足部距离	惩罚足部距离过近的情况。	-10.0

实现细节

- Control period: `ctrl_dt` $= 0.02$ s; simulation step: `sim_dt` $= 0.002$ s.
- External pushes enabled with interval 5.0-10.0s and magnitude 0.1-1.0N.
- Enhanced tracking rewards with separate x/y linear velocity tracking.
- Sophisticated foot kinematics penalties including roll, yaw, and swing phase rewards.

Table 5: Domain randomization parameters (Note: $U[a, b]$ denotes uniform distribution over $[a, b]$, $\pm U(c)$ denotes uniform distribution over $[-c, c]$)

Parameter	**Range**	**Description**
Floor friction	$U[0.2, 0.6]$	Ground friction coefficient
Joint friction loss	$\times U[0.9, 1.1]$	Joint friction scaling
Joint armature	$\times U[1.0, 1.05]$	Joint armature scaling
Link masses	$\times U[0.98, 1.02]$	Body mass scaling
Torso mass	$+U[-1.0, 1.0]$ kg	Additional torso mass
Initial joint positions	$\pm U[0.05]$ rad	Default pose randomization
Joint stiffness	$\times U[0.7, 1.3]$	Actuator gain scaling
Joint damping	$\times U[0.7, 1.3]$	Damping coefficient scaling
Ankle damping	$\times U[0.5, 2.0]$	Higher range for ankle joints

C.3 Finetuning Environments

C.3.1 T1LowDimJoystick (Finetune)

(`T1LowDimJoystick` as an example to illustrate the finetuning environment setup, `G1LowDimJoystick` is omitted for brevity.)

Task summary. This is a simplified version of the T1LowDimJoystick environment designed specifically for sim-to-sim transfer from MuJoCo Playground to Brax and finetuning in Brax. The environment ensures that policies trained in MuJoCo Playground can be successfully transferred to Brax with similar reward scales and performance characteristics. The reward settings are adopted from Booster Gym (Wang et al., 2025).

Key modifications for finetuning:

- **Normalized observations** using predefined limits for consistent scaling across simulators
- **Simplified reward structure** with many terms zeroed out to focus on essential behaviors
- **Continuous gait phase** representation using gait process and frequency

State (47D, normalized). The normalized state consists of gravity vector (3), gyroscope readings (3), joystick command (v_x, v_y, ω) (3), gait phase cosine/sine (2), joint angles (12), joint velocities (12), and previous action (12). All observations are normalized to $[-1, 1]$ using predefined limits.

Privileged state (87D, normalized). Includes base state plus raw gyroscope, gravity vector, orientation quaternion (4), base velocity, joint positions/velocities, root height, gait process, and gait frequency.

Observation normalization: Observations are normalized using:

$$\text{obs_normalized} = \frac{2 \times (\text{obs} - \min)}{\max - \min} - 1$$

with predefined limits for each observation dimension.

Action space. Motor targets are computed using a PD control law:

$$u = K_p(a + q_{\text{default}} - q) + K_d(0 - \dot{q}) \tag{24}$$

where u is the output motor torque, K_p and K_d are the proportional and derivative gains, respectively, a is the action output from the policy q_{default} is a nominal joint position, q is the current joint position, and $\dot{q}$ is the current joint velocity.

- 控制周期：ctrl_dt $= 0.02$s；仿真步长：sim_dt $= 0.002$s。
- 启用外部推力，间隔为 5.0-10.0s，大小为 0.1-1.0N。
- 增强的跟踪奖励，包含独立的 x/y 线速度跟踪。
- 复杂的足部运动学惩罚，包括翻滚、偏航和摆动相奖励。

表 5：域随机化参数（注： $U[a, b]$ 表示在 $[a, b]$，$\pm U$上的均匀分布；(c) 表示在 $[-c, c]$上的均匀分布）

参数	**范围**	**描述**
地面摩擦	$U[0.2, 0.6]$	地面摩擦系数
关节摩擦损耗	$\times U[0.9, 1.1]$	关节摩擦缩放
关节骨架	$\times U[1.0, 1.05]$	关节骨架缩放
连杆质量	$\times U[0.98, 1.02]$	刚体质量缩放
躯干质量	$+U[-1.0, 1.0]$ kg	附加躯干质量
初始关节位置	$\pm U[0.05]$ rad	默认姿态随机化
关节刚度	$\times U[0.7, 1.3]$	执行器增益缩放
关节阻尼	$\times U[0.7, 1.3]$	阻尼系数缩放
脚踝阻尼	$\times U[0.5, 2.0]$	脚踝关节的更高范围

C.3 微调环境

C.3.1 T1LowDimJoystick（微调）

（以 T1LowDimJoystick 为例来说明 微调 环境设置，G1LowDimJoystick 为简洁起见已省略。）

任务摘要。这是 T1LowDimJoystick 环境的简化版本，专为从 MuJoCo Playground 到 Brax 的仿真到仿真迁移以及在 Brax 中的微调而设计。该环境确保在 MuJoCo Playground 中训练的策略能够成功迁移到 Brax，并具有相似的奖励尺度和性能特征。奖励设置采纳自 Booster Gym（Wang 等，2025）。

微调的关键修改：

- **归一化观测**使用预定义限值，以便在不同仿真器之间实现一致缩放
- **简化奖励结构**通过将许多项设为零，以专注于基本行为
- **连续步态相位**表示，利用步态过程和频率

状态（47 维，已归一化）。归一化状态包括重力向量（3）、陀螺仪读数（3）、操纵杆指令（v_x，v_y，ω）（3）、步态相位余弦/正弦（2）、关节角度（12）、关节速度（12）以及上一动作（12）。所有观测均使用预定义限值归一化至 $[-1, 1]$。

特权状态（87 维，已归一化）。包括基础状态以及原始陀螺仪、重力向量、方向四元数（4）、基座速度、关节位置/速度、根高度、步态过程和步态频率。

观测归一化：观测通过以下方式进行归一化：

$$\text{obs_normalized} = \frac{2 \times (\text{obs} - \min)}{\max - \min} - 1$$

每个观测维度均设有预定义限值。

动作空间。电机目标通过 PD 控制律计算得出：

$$u = K_p(a + q_{\text{default}} - q) + K_d(0 - \dot{q}) \tag{24}$$

其中 u 是输出电机扭矩，K_p 和 K_d 分别是比例和微分增益，a 是来自策略的动作输出，q_{default} 是标称关节位置，q 是当前关节位置，$\dot{q}$ 是当前自由度速度。

Table 6: Finetune reward terms (Note: many terms disabled for sim-to-sim transfer)

Term	Description	Scale
survival	Survival bonus per step.	0.25
tracking_lin_vel_x	Track commanded v_x velocity.	1.0
tracking_lin_vel_y	Track commanded v_y velocity.	1.0
tracking_ang_vel	Track commanded yaw rate.	2.0
base_height	Reward proximity to target height (0.65m).	0.2
orientation	Penalize torso tilt from vertical.	-5.0
feet_swing	Reward proper foot swing phase.	3.0
feet_slip	Penalize slipping contacts.	-0.1
feet_yaw_diff	Penalize difference in foot yaw angles.	-1.0
feet_yaw_mean	Penalize deviation from base yaw.	-1.0
feet_roll	Penalize foot roll angles.	-1.0
feet_distance	Penalize feet being too close.	-10.0

Reward terms (simplified for transfer):

Disabled reward terms: torques, torque_tiredness, power, lin_vel_z, ang_vel_xy, dof_vel, dof_acc, root_acc, action_rate, dof_pos_limits, collision, feet_vel_z (set to 0.0 scale).

Implementation details

- External pushes enabled (interval: 5.0-10.0s, magnitude: 0.1-1.0N)
- Normalized observations ensure consistent scaling across simulators

Table 7: Domain randomization parameters (Note: $U[a, b]$ denotes uniform distribution over $[a, b]$, $\pm U(c)$ denotes uniform distribution over $[-c, c]$)

Parameter	Range	Description
Floor friction	$U[0.2, 0.6]$	Ground friction coefficient
Joint friction loss	$\times U[0.9, 1.1]$	Joint friction scaling
Joint armature	$\times U[1.0, 1.05]$	Joint armature scaling
Link masses	$\times U[0.98, 1.02]$	Body mass scaling
Torso mass	$+U[-1.0, 1.0]$ kg	Additional torso mass
Initial joint positions	$\pm U[0.05]$ rad	Default pose randomization
Joint stiffness	$\times U[0.7, 1.3]$	Actuator gain scaling
Joint damping	$\times U[0.7, 1.3]$	Damping coefficient scaling
Ankle damping	$\times U[0.5, 2.0]$	Higher range for ankle joints

D LARGE LANGUAGE MODEL USAGE

We gratefully acknowledge the use of ChatGPT and DeepSeek to assist in polishing, refining, and condensing the text of this paper.

表 6：微调奖励项（注：许多项在仿真到仿真迁移中已禁用）

Term	描述	缩放
生存	每步生存奖励。	0.25
跟踪线速度 x	跟踪指令 v_x 速度。	1.0
跟踪线速度 y	跟踪指令 v_y 速度。	1.0
跟踪角速度	跟踪指令偏航率。	2.0
基座高度	奖励接近目标高度（0.65 米）。	0.2
朝向	惩罚躯干偏离垂直方向的倾斜。	-5.0
足部摆动	奖励正确的足部摆动相位。	3.0
足部打滑	惩罚滑动接触。	-0.1
足部偏航角差	惩罚足部偏航角的差异。	-1.0
足部偏航角均值	惩罚与基座偏航角的偏差。	-1.0
足部横滚角	对脚部滚动角度进行惩罚。	-1.0
足部距离	对足部距离过近进行惩罚。	-10.0

奖励项（为迁移简化）：

禁用的奖励项： 扭矩、扭矩疲劳、功率、z 轴线速度、xy 轴角速度、自由度速度、自由度加速度、根节点加速度、动作速率、自由度位置限制、碰撞、z 轴脚速度（缩放设为 0.0）。

实现细节

- 已启用外部推力（间隔：5.0-10.0 秒，大小：0.1-1.0 牛）
- 归一化观测可确保跨仿真器的一致缩放

表 7：域随机化参数（注：$U[a, b]$ 表示在 $[a, b]$，$\pm U$ 上的均匀分布，(c) 表示在 $[-c, c]$ 上的均匀分布）

参数	范围	描述
地面摩擦	$U[0.2, 0.6]$	地面摩擦系数
关节摩擦损耗	$\times U[0.9, 1.1]$	关节摩擦缩放
关节骨架	$\times U[1.0, 1.05]$	关节骨架缩放
连杆质量	$\times U[0.98, 1.02]$	刚体质量缩放
躯干质量	$+U[-1.0, 1.0]$ kg	附加躯干质量
初始关节位置	$\pm U[0.05]$ rad	默认姿态随机化
关节刚度	$\times U[0.7, 1.3]$	执行器增益缩放
关节阻尼	$\times U[0.7, 1.3]$	阻尼系数缩放
脚踝阻尼	$\times U[0.5, 2.0]$	脚踝关节的更高范围

D 大型语言模型使用情况

我们衷心感谢使用 ChatGPT 和 DeepSeek 协助润色、完善并精简本文内容。